A NOVEL CRYPTOGRAPHIC SCHEME BASED ON THE COLLATZ CONJECTURE

A PROJECT REPORT SUBMITTED BY

R. M. V. V. BANDARA (S/16/322)

to the

Department of Mathematics

in partial fulfillment of the requirement for the award of the degree

of

BACHELOR OF SCIENCE

of the

University of Peradeniya

Sri Lanka

August 2022

Declaration

I do hereby declare that the work reported in this research report was exactly carried out by me under the supervision of Dr.P.G.R.S. Ranasinghe. It describe the result of my own independent work due reference has been made in the text. No part of this research report has been submitted earlier or concurrently for the same any other degree.

R.M.V.V.Bandara	Date

Certificated by,

Dr. P.G.R.S. Ranasinghe (Supervisor)	Date

Dr. T. H. K. R. De Silva (Head of the department)	Date

Abstract

The Collatz conjecture is an unsolved problem in mathematics, which is named after the mathematician Lothar Collatz, who introduced the idea in 1937.

First part of this research, we have approached the Collatz conjecture in a constructive way. In the constructed structure, the Collatz sequence converges upwards and we have defined a function g for any odd number, which diverges downward. For every term in the divergent sequence generated from the function g, we have a convergent sequence generated from the Collatz function. We have showed the function g is divergent for any positive odd integer. Thus, our present study provides a good justification for why the Collatz conjecture holds true for all positive integers.

As the second part of this research, we used Python codes to generate $Pn+1$ sequences starting from the numbers 1 to 1000,000 and we checked for the numbers which are repeating in a generated sequence and the length (number of terms it takes to reach a loop). Since all the sequences had a finite length, we can conclude that all $Pn + 1$ sequences are convergent to a loop for $P < 50$, but the sequences which are similar to the Collatz sequence, have only one loop. For the first 4 Pell-prime numbers, we had only one loop and also, since there are infinitely many Pell-prime numbers, there can be infinitely many similar sequences to the Collatz sequence.

As the third part of this research, we have proposed a new symmetric cryptosystem in which the sender and the receiver hold the secret key pair (k, l) which has been chosen randomly. A plaintext is encrypted using a large, odd number k, and l number of iterations in the Collatz sequence starting with k. The Collatz sequence is a chaos-based sequence and for every positive integer there is a unique mapping, and a slight change of the number makes a huge difference in the generated sequence. Thus, the chaotic nature makes this cryptosystem more secure. This cryptosystem has uncomplicated transformations which are very simple to be analyzed and is efficient because the only operations involved are multiplication and addition. It can be used to build more secure cryptosystems by repeating the encryption process or implementing it with other cryptosystems.

Key words: Chaotic nature, Collatz sequence, Cryptography

Dedication

To

Dr. P.G.R.S. Ranasinghe

I beg to dedicate it to your name. You have always judged me liberally; Your encouragement has enriched my soul and inspired me to pursue and complete this research.

Acknowledgements

This project would not have been possible without the kind support and help of many individuals.

First and foremost, I would like to thank my research supervisor Dr. P.G.R.S. Ranasinghe, Senior Lecturer in the Department of Mathematics for his guidance, supervision, and persistent help throughout this research. Also, I would like to pay my gratitude to my friend, Mr. Rajitha Gunathilake from the Faculty of Information Technology, University of Moratuwa for helping me thorough the Python coding part of this research.

Finally, I would like to thank all my colleagues for their valuable feedback and friendship.

Contents

Declaration **i**

Abstract **ii**

Acknowledgements **iv**

1 Introduction **1**

1.1 Mathematical Background 1

1.1.1 The Collatz Conjecture 1

1.1.2 Research areas 2

1.2 History and Evaluation of Cryptography 6

1.2.1 The Ancient Roots of Cryptography 6

1.2.2 Developments in the Middle Ages and Renaissance 7

1.2.3 Advances in More Recent Centuries 8

1.2.4 Cryptography in the Computer Age 8

1.3 General Cryptography 9

1.3.1 Cryptographic Goals 9

1.3.2 Classifications of Cryptography 10

1.3.3 Symmetric-key Cryptographic systems 10

2 Literature Survey **12**

2.1 The $\mathbf{3x+1}$ Problem[21] 13

2.1.1 $\mathbf{3x+1}$ Conjecture (First form) 14

2.1.2 $\mathbf{3x+1}$ Conjecture (Second form) 15

2.2 Unsolved problems formulated from $\mathbf{3x+1}$ problem[21] 15

3 Materials and Methods **16**

3.1 The Collatz function and the conjecture 16

3.2 Connected Sub-Sequences 17

3.3 Sequences connected to $\{1 \cdot 2^m\}$ sequence 19

3.4 Odd number sequences 19

3.5 Infinite loops of sequences and the inverse Collatz function 21

3.5.1 Infinite loops of sequences 21

3.5.2 The inverse Collatz function 22

3.5.3 The sequences generated by the function g are divergent for all positive odd integers 23

3.6 Odd numbers of the form $\mathbf{6n-1}$ and $\mathbf{6n+1}$ 26

3.6.1 Odd numbers of the form $\mathbf{6n+1=2^l3^mq+1}$ 27

3.6.2 Odd numbers of the form $\mathbf{6n-1=2^l3^mq-1}$ 27

4 Results **28**

4.1 Similar Sequences to the Collatz sequence 28

4.1.1 **Pn + 1** Sequence lengths and loops 29

4.1.2 **Pn + 1** functions with Pell prime numbers 40

4.2 Encryption and Decryption algorithm using the Collatz sequence 41

4.2.1 Encryption Algorithm 41

4.2.2 Decryption Algorithm Using the Collatz sequence 43

5 Conclusions **45**

5.1 The approach on the Collatz conjecture 45

5.2 Similar sequences to the Collatz sequence 46

5.3 The cryptographic scheme based on the Collatz sequence 46

Appendices **47**

Appendix A: Python Encryption Algorithm 47

Appendix B: Python Decryption Algorithm 49

Appendix C: Python codes for number of steps in $Pn + 1$ Functions 51

Bibliography **52**

List of Figures

2.1 A portion of the Collatz graph 13

3.1 Visual diagram of connected sub-sequences(The Collatz structure) 18

4.1 Number of terms up to 1, in the sequences generated by the $3n + 1$ function using Python. 29

4.2 Number of terms up to 1, in the sequences generated by the $5n + 1$ function using Python. 30

4.3 Number of terms up to 1, in the sequences generated by the $7n + 1$ function using Python. 30

4.4 Number of terms up to a loop, in the sequences generated by the $11n+1$ function using Python. 31

4.5 Number of terms up to a loop, in the sequences generated by the $13n+1$ function using Python. 31

4.6 Number of terms up to a loop, in the sequences generated by the $17n+1$ function using Python. 32

4.7 Number of terms up to a loop, in the sequences generated by the $19n+1$ function using Python. 33

4.8 Number of terms up to a loop, in the sequences generated by the $23n+1$ function using Python. 34

4.9 Number of terms up to 1, in the sequences generated by the $29n + 1$ function using Python. 34

4.10 Number of terms up to a loop, in the sequences generated by the $31n+1$ function using Python. 35

4.11 Number of terms up to a loop, in the sequences generated by the $37n+1$ function using Python. 36

4.12 Number of terms up to 1, in the sequences generated by the $41n + 1$ function using Python. 37

4.13 Number of terms up to a loop, in the sequences generated by the $43n+1$ function using Python. 38

4.14 Number of terms up to a loop, in the sequences generated by the $47n+1$ function using Python. 39

Chapter 1

Introduction

1.1 Mathematical Background

1.1.1 The Collatz Conjecture

Lothar Collatz, a German mathematician, proposed a conjecture in 1937 stating that the following sequence will always reach 1, regardless of the given positive integer n: if the previous term in the sequence is even, the next term is half of the previous term; if the previous term in the sequence is odd, the next term is three times the previous term plus one [3]. Given modern mathematics, this conjecture is thought to be impossible to prove. Indeed, Paul Erdős asserted that "mathematics may not be prepared for such problems" [11]. This sentiment was echoed by Jeffrey Lagarias, who wrote a thorough summary of numerous findings pertaining to the conjecture in [21].

Despite the apparent inability to prove the Collatz conjecture today, several papers have detailed discoveries related to the seemingly impossible proof. Terence Tao, perhaps the most notable recent development, demonstrated that most Collatz map orbits achieve nearly bounded values [37]. The College Mathematics Journal published an excellent review of his paper [12]. This significant development sheds light on one potential avenue for proving the Collatz conjecture; however, other avenues for proof remain open.

The Collatz conjecture has been broken up into several smaller problems or rephrased by some

mathematicians in an effort to make it more manageable. Ren [31] demonstrated, for instance, that the Collatz conjecture can be proved if his reduced Collatz conjecture (RCC) is validated. Ren also established that the lower half of the natural numbers exhibit reduced Collatz dynamics; however, the RCC's second half of the proof is still open [30]. While investigating reduced Collatz dynamics might be a promising path for proving the Collatz conjecture, it is probably advantageous to suggest other possible paths [4],[41].

Several papers [20],[22],[6] have detailed the various unsuccessful methods that mathematicians have explored to solve the question. Perhaps the proof of this mesmerizing conjecture is so desired because the conjecture itself finds use in various applications. The Collatz conjecture is used in high-uncertainty audio signal encryption [32], image encryption [5], dynamic software watermarking [24], and information discovery [13]. Although the lack of a proof does not stop the Collatz hypothesis from being used, mathematical phenomena connected to the conjecture may be of interest in other applications, such as cryptography or information theory.

1.1.2 Research areas

The Collatz conjecture is applicable to many branches of mathematics. The issue has been studied in six key areas:

1. Number theory: Analysis of periodic orbits of the map;
2. Dynamical systems: Behavior of generalizations of the $3m + 1$ map;
3. Ergodic theory: Invariant measures for generalized maps;
4. Theory of computation: Undecidable iteration problems;
5. Stochastic processes and probability theory: Models yielding heuristic predictions for the behavior of iterates;
6. Computer science: Algorithms for computing iterates and statistics, and explicit computations. We treat these in turn.

Number Theory

The $3x + 1$ problem is an arithmetic problem, hence it belongs to elementary number theory. This is the direct connection to number theory. R. K. Guy [11] [Problem E16] does classify it as an unsolved problem in number theory. Exploring the cycles of the $3x + 1$ map produces exponential Diophantine equation-related issues. The significant work on linear forms in logarithms by Baker and Masser-Wustholz provides information on the non-existence of cycles with different lengths and specific patterns of even and odd iterates. In a number theory framework, a class of generalized $3x + 1$ functions has been defined, in which operations on the domain of integers are replaced by operations on the ring of integers of an algebraic number field, or by function field analogues like a polynomial ring with coefficients in a finite field. Also, the pieces by Lagarias [19] and Chamberland [7] discuss number-theoretic results.

Dynamical Systems

The theory of discrete dynamical systems is concerned with the behavior of functions during iteration, whereas the theory of continuous dynamical systems is concerned with flows or solutions to differential equations. Because the $3x + 1$ problem may be thought of as iterating a map, it is a discrete dynamical system on the state space $\mathbb{Z}$. This point of view was expressed in Wirsching [40]. The composition of functions is a crucial procedure for iteration. Lausch and Nobauer [23] [Chap. 4.5] show how to frame iteration and composition concerns in the context of universal algebra. The classes of generalized $3x+1$ functions and quasi—polynomial functions are each closed in the taxonomy above under addition and composition of functions. In relation to the $3x + 1$ problem and the theory of computation, the iteration qualities of the first three classes of aforementioned functions have been explored. However, there is still much to learn about the iteration of broad quasi-polynomial functions.

This approach of looking at the problem suggests that it would be beneficial to get dynamical systems on larger domains, such as the real numbers $\mathbb{R}$ and the complex numbers $\mathbb{C}$, in order to examine the $3x+1$ function. Other extensions involve establishing comparable functions on the ring $\mathbb{Z}_2$ of 2—adic integers or, for generalized $3x + 1$ mappings, on a ring of d—adic integers,

for a value of 0! determined by the function. A wide range of behaviors are possible when generalized $3x + 1$ functions are considered over larger domains. The papers by Chamberland [7] and Matthews [25] discuss these subjects. See Akin [2] for a broad foundation on topological dynamics.

Ergodic Theory

The connection to ergodic theory develops as a result of the dynamical systems perspective, but it also includes the requirement of an invariant measure. Early on, it was discovered that there are finitely additive measures that the $3x + 1$ map on integers preserves. Maps that are invariant under standard measures (countably additive measures) result from the extension of generalized $3x + 1$ functions to deadic integers. One such instance is the (unique continuous) extension of the $3x+1$ map to the 2-adic integers, which has the 2-adic measure as an invariant measure and is ergodic with regard to this measure. Topics related to ergodic theory are covered in Matthews' [25] and Kontorovich and Lagarias' [17] examinations. Classifying all invariant measures for generalized $3x+1$ functions on the d—adic integers is an intriguing open problem.

Mathematical Logic and the Theory of Computation

The relationship to logic and computation theory begins with Conway's discovery of a generalized $3x + 1$ function whose iteration can simulate a universal computer. Conway [8] demonstrated an unsolvable iteration issue for a specific generalized $3m + 1$ function: beginning with a given input that is a positive integer n, determine whether or not any iterate of this map with this input is ever a power of 2. In this regard, the $3x+1$ problem can be phrased as saying that, beginning with any positive integer n, some iterate $C^{(k)}(n)$ of the Collatz function is a power of 2. It turns out that iteration of $3x + 1$-like functions was already explored in understanding the strength of some logical theories as early as the late 1960s; these involved partially specified functions taking integers to integers (with undefined output for some integers), see Isard and Zwicky [14]. Such functions have recently emerged as a result of research into the processing power of "small" Turing machines, which are too small to encode a universal computer. These topics are covered in Michel and Margenstern's paper [26].

Probability Theory and Stochastic Processes

When attempting to model the behavior of the $3x + 1$ iteration on huge sets of integers, a relationship to probability theory and stochastic processes emerges. This results in heuristic probabilistic models for the iteration that can anticipate its behavior. Some authors have proposed that iteration can be considered as a type of pseudo-random number generator, with the input given by a probability distribution and the question of how this probability distribution evolves during iteration. In the other approach, trees of inverse iterates can be studied (the inverse map is many—to—one, resulting in a unary-binary tree of inverse iterates). The structure of these trees, whose root node is an integer selected at random from a probability distribution, can be inquired about here. This can be modeled using a stochastic model, such as a branching random walk, which corresponds to random tree growth. The work by Kontorovich and Lagarias [17] covers these subjects in detail.

Computer Science: Machine Models, Parallel and Distributed Computation

Conway [9] established the Fractran model of computation as a universal computer model in 1987, based on his earlier work on the $3x + 1$ problem. This computational paradigm is similar to Marvin Minsky's register machine (or counter machine) model [28], [27][Sect. 11.1]. Both of these machine models have lately been recognized as relevant for creating computational models using chemical reaction networks, as well as for biological computing, see Soloveichik et al [35] and Cook et al [10]. The requirement to conduct computer experiments to test the $3x + 1$ conjecture and to investigate various aspects and patterns of the $3x + 1$ repetition raises additional computational issues. One research topic is to design efficient methods for large-scale computation utilizing either parallel processors or a distributed computer system. The $3x + 1$ supposition has been tested to a very large value of n. The mechanism employed in [29] to obtain record results is parallelizable. Roosendaal [1] describes several large-scale computations for the $3x + 1$ problem that used distributed computing.

[1] `http://www.ericr.nl/wondrous/`

1.2 History and Evaluation of Cryptography

Cryptography, the science of creating codes and encryption methods for secure communication, is one of the most critical components of modern cryptocurrencies and blockchains. Today's cryptographic techniques, on the other hand, are the result of an incredibly long history of development. Cryptography has been used to securely transmit information since ancient times. The fascinating history of cryptography that has led up to the advanced and sophisticated methods used for modern digital encryption is detailed below during four time periods.

- The Ancient Roots of Cryptography
- Developments in the Middle Ages and Renaissance
- Advances in More Recent Centuries
- Cryptography in the Computer Age

1.2.1 The Ancient Roots of Cryptography

Primitive cryptographic techniques are known to have existed in ancient times, and cryptography appears to have been used to some extent by most early civilizations. The most basic form of cryptography, symbol replacement, appears in both ancient Egyptian and Mesopotamian writings. The earliest known example of this type of cryptography was discovered in the tomb of Khnumhotep II, an Egyptian noble who lived around 3,900 years ago.

The goal of symbol replacement in the Knhumhotep inscription was to improve its linguistic appeal, not to conceal information. The first known use of cryptography to protect sensitive information was around 3,500 years ago, when a Mesopotamian scribe used cryptography to conceal a formula for pottery glaze, which was used on clay tablets.

Cryptography was widely used to protect important military information by later periods of antiquity, and it still serves that purpose today. Messages were encrypted in the Greek city-state of Sparta by being written on parchment and laid over a cylinder of a specific size, rendering

the message indecipherable until the recipient wrapped it around a similar cylinder. Similarly, coded messages were used by spies in ancient India as early as the 2nd century BC.

The Romans may have developed the most advanced cryptography in the ancient world. The Caesar cipher was a well-known example of Roman cryptography that involved shifting the letters of an encrypted message a certain number of places down the Latin alphabet. A recipient could successfully decode the otherwise illegible message if they knew this system and the number of places to shift the letters.

1.2.2 Developments in the Middle Ages and Renaissance

Throughout the Middle Ages, cryptography grew in importance, but substitution ciphers, such as the Caesar cipher, remained the standard. Cryptanalysis, the science of cracking codes and ciphers, began to catch up to cryptography, which was still a relatively primitive science. Around 800 AD, Al-Kindi, a well-known Arab mathematician, developed a technique known as frequency analysis, which rendered substitution ciphers vulnerable to decryption. For the first time, people attempting to decipher encrypted messages had access to a systematic method for doing so, necessitating further advancements in cryptography in order to remain useful.

Leone Alberti invented the polyalphabetic cipher in 1465, which is regarded as a solution to Al-frequency Kindi's analysis technique. A message is encoded using two distinct alphabets in a polyalphabetic cipher. The first is the original message's alphabet, while the second is an entirely different alphabet in which the message appears after it has been encoded. Polyalphabetic ciphers, when combined with traditional substitution ciphers, greatly increased the security of encoded information. The frequency analysis technique was useless unless the reader knew the alphabet in which the message was originally written.

During the Renaissance period, new methods of encoding information were also developed, including a popular early method of binary encoding invented in 1623 by the noted polymath Sir Francis Bacon.

1.2.3 Advances in More Recent Centuries

Throughout the centuries, the science of cryptography advanced gradually. In the 1790s, Thomas Jefferson described, but perhaps never built, a major breakthrough in cryptography. His cipher wheel invention consisted of 36 rings of letters on moving wheels that could be used to achieve complex encoding. This concept was so advanced that it would serve as the foundation for American military cryptography until the end of World War II.

During World War II, the Enigma machine provided a perfect example of analog cryptography. This device, like the wheel cipher, was used by the Axis powers to encode a message, making it nearly impossible to read without another Enigma. Early computer technology was eventually used to aid in the breakage of the Enigma cipher, and successful decryption of Enigma messages is still regarded as a critical component of the eventual Allied victory.

1.2.4 Cryptography in the Computer Age

With the advent of computers, cryptography advanced far beyond what it was in the analog era. Many sensitive devices and computer systems now use 128-bit mathematical encryption, which is far stronger than any ancient or medieval cipher. Beginning in 1990, computer scientists were working on an entirely new type of cryptography known as quantum cryptography in the hopes of increasing the level of security provided by modern encryption.

Cryptographic techniques have recently been used to make cryptocurrencies possible. Cryptocurrencies use advanced cryptographic techniques such as hash functions, public-key cryptography, and digital signatures. These methods are primarily used to secure data stored on blockchains and to authenticate transactions. An Elliptical Curve Digital Signature Algorithm (ECDSA) is a type of cryptography that underpins Bitcoin and other cryptocurrency systems to provide extra security and ensure that funds can only be used by their rightful owners.

Cryptography has advanced significantly over the last 4,000 years, and it is unlikely to slow down anytime soon. Cryptography will continue to advance as long as sensitive data needs to be protected. Although the cryptographic systems used in cryptocurrency blockchains today represent some of the most advanced forms of this science, they are also part of a long tradition.

1.3 General Cryptography

Cryptography is the process of converting data from readable to unreadable form to meet security needs. Cryptography or encryption also provides user authentication while simultaneously protecting data. The original data is usually referred to as plaintext, while encrypted data is referred to as ciphertext. So the process of converting plaintext to ciphertext is known as encryption, while the process of converting ciphertext to plaintext is known as decryption [36]. Because the user must transfer his specific data across the internet to be stored in the cloud, cryptography is critical in cloud computing [15]. As a result, without encryption, the data will be easily detected by attackers; similarly, in cloud storage, the data must be protected against attackers using strong encryption [33].

1.3.1 Cryptographic Goals

There are four goals that are almost always detected and prevent attackers from altering, tampering with, stealing, and so on [34],[1]. These objectives are:

- **Confidentiality:**

  We guarantee that only those with permission may access data. There are several ways to guarantee confidentiality, including physical security and encryption algorithms that give data an ambiguous structure.

- **Data integrity:**

  It is a procedure used to guarantee that data will be transferred accurately and without being altered. Data may be changed by replacement, deletion, and insertion.

- **Authentication:**

  It is the process that assists the recipient in determining whether the data received came from the true sender only.

- **Non-repudiation:**

  It is the process of ensuring that both the sender and the recipient do not refuse the

transmission, which means that the sender cannot deny transmitting the data and the recipient cannot deny receiving the data .

1.3.2 Classifications of Cryptography

There are three types of cryptography systems: symmetric cryptography, asymmetric cryptography, and hash functions [33]. Briefly,

1. **Symmetric cryptography:** This is a technique for converting plaintext to ciphertext by using the same key on both sides (sender/recipient). AES, DES, Blowfish, and RC5 are examples of symmetric cryptography algorithms.

2. **Asymmetric cryptography:** Two keys are used in asymmetric cryptography, also known as public key cryptography. To encrypt data, the sender will use the public key of the recipient, which is known to all network users; to decrypt data, the private key of the recipient, which is known only to the recipient, will be used to convert ciphertext to original plaintext.

3. **Hash function:** It uses mathematical transformations to encrypt data irreversibly. SHA-1, MD5, and other hash functions are examples. Because ciphertext cannot be converted to plaintext, the hash function lacks keys.

1.3.3 Symmetric-key Cryptographic systems

The same single key is used by symmetric encryption algorithms to encrypt and decrypt data. This kind of algorithm is used by many, including Blowfish, DES, AES, and others. Each algorithm employs a unique technique to encrypt and decrypt the data. Additionally, each of them will encrypt and decrypt a block of fixed size of data and a key that is also fixed in size. Such algorithms restrict the input of plaintext to English alphabets, specific symbols, and numerical values only. As a result, the output (ciphertext) is a document made up of special

characters, alphabets, numbers, or a combination of all three. We can list the symmetric encryption components as follows [33] :

1. Plaintext: This is the original data that the sender wishes to send to a specific recipient. These values will be fed into the encryption algorithm.

2. Encryption algorithm: A set of processes that will execute into plaintext with the help of a secret key to produce ciphertext.

3. Secret key: A value that is used to combine plaintext in order to transform it to ciphertext; this value is independent of plaintext.

4. Ciphertext: The output of the original plaintext after it has been encrypted. It will be vastly different from plaintext.

5. Decryption algorithm: A set of processes that will be executed on ciphertext with the help of a secret key to produce original plaintext.

Chapter 2

Literature Survey

The precise origin of the $3n + 1$ problem is unknown. For many years, it spread by word of mouth in the mathematical community. Lothar Collatz of the University of Hamburg is traditionally credited with introducing the problem. In the 1930s, he became interested in number-theoretic functions after hearing lectures by Edmund Landau, Oskar Perron, and Issai Schur. His interest in graph theory led him to the idea of representing such number-theoretic functions as directed graphs, and questions about the structure of such graphs are related to the behavior of such function iterates [38]. He considered the function in his notebook dated July 1, 1932.

$$g(n) = \begin{cases} \frac{2}{3}n, & \text{If } n \equiv 0 \pmod 3 \\ \frac{4}{3}n - \frac{1}{3}, & \text{If } n \equiv 1 \pmod 3 \\ \frac{4}{3}n + \frac{1}{3}, & \text{If } n \equiv 2 \pmod 3 \end{cases}$$

Which gives rise to a permutation P of the natural numbers,

$$P = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & \\ 1 & 3 & 2 & 5 & 7 & 4 & 9 & 11 & 6 & \cdots \end{pmatrix}$$

He posed the problem of determining the cycle structure of P, specifically whether or not the cycle of this permutation containing 8 is finite or infinite, i.e., whether or not the iterates $g^k(8)$ remain bounded or are unbounded [21]. The initial Collatz problem is the study of the iterates of g(n). Collatz distributed his iteration problems at the International Congress of

Mathematicians in Cambridge, Massachusetts in 1950, despite the fact that he never published any of them. Eventually, the original Collatz problem was printed as a result. His initial query about $g^k(8)$ has never been resolved, and the cycle it belongs to is thought to be endless. The 3m + 1 problem, which was first discovered by B. Thwaites in 1952 [20], was undoubtedly well-known to the mathematical community by the early 1950s, regardless of its precise origins.

2.1 The 3x + 1 Problem[21]

Let $\mathbb{N}$ be the set of all positive integers and $n \in \mathbb{N}$. Collatz defined the following map:

$$T(n) = \begin{cases} \frac{n}{2}, & \text{If n is even} \\ 3n+1, & \text{If n is odd} \end{cases}$$

A sequence can be formed by iterating over $T : n, T(n), T^2(n) = T(T(n)), ..., T^k(n) = T(T^{k-1}(n))$ for $k \in \mathbb{N}$. The Collatz conjecture asserts that the sequence obtained by iterating T always reaches integer 1, no matter which positive integer n begins the sequence. One way to think about the $3x+1$ problem is as a directed graph with positive integer vertices and directed edges from n to T. In honor of L.Collatz [38], we call this graph the Collatz graph of T(n). The figure depicts a section of the Collatz graph of T(n).

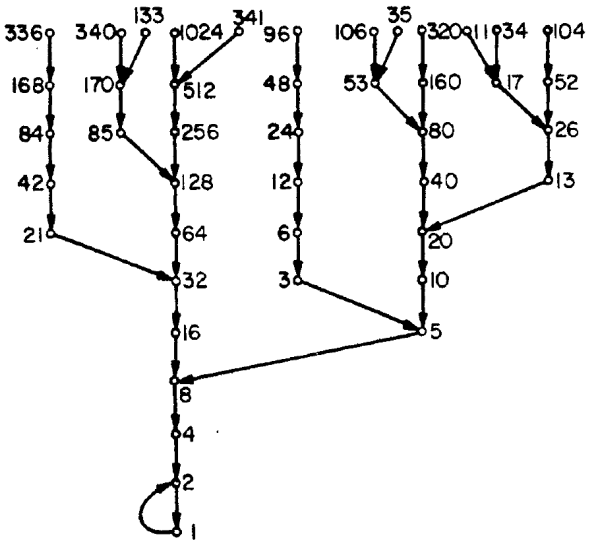


Figure 2.1: A portion of the Collatz graph

A directed graph is said to be **weakly connected**, if it is connected when viewed as an undirected graph, i.e., for any two vertices there is a path of edges joining them, ignoring the directions on the edges. The $3x+1$ Conjecture can be formulated in terms of the Collatz graph as follows.

2.1.1 3x + 1 Conjecture (First form)

The Collatz graph of T(n) on the positive integers is weakly connected. We call the sequence of iterates $(n, T(n), T^{(2)}(n), T^{(3)}(n), ...)$ the trajectory of n. There are three possible behaviors for such trajectories when $n > 0$.

1. Convergent trajectory. Some $T^{(k)}(n) = 1$. This happens when the sequence generated by T, for some positive integer n is convergent to 1.
2. Non-trivial cyclic trajectory. The sequence $T^{(k)}(n)$ eventually becomes periodic and $T^{(k)}(n) \neq 1$ for any $k \geq 1$. This happens when the sequence generated by T for some positive integer n has a loop other than the trivial loop containing 1.
3. Divergent trajectory. $\lim_{k\to\infty} T^{(k)}(n) = \infty$. This happens when the sequence generated for some positive integer n is not convergent.

The $3x + 1$ Conjecture asserts that all trajectories of positive n are convergent. It is certainly true for $n > 1$ that $T^{(k)}(n) = 1$ cannot occur without some $T^{(k)}(n) \leq n$ occurring. Call the least positive k for which $T^{(k)}(n) \leq n$ the *stopping time* $\sigma(n)$ of n, and set $\sigma(n) = \infty$ if no k occurs with $T^{(k)}(n) \leq n$. Also call the least positive k for which $T^{(k)}(n) = 1$ the *total stopping time* $\sigma_\infty(n)$ of n, and set $\sigma_\infty(n) = \infty$ if no such k occurs. i.e,
Stopping time $\sigma(n)$ =: Number of steps on the Collatz sequence of n until $T^{(k)}(n) \leq n$.
Total stopping time $\sigma_\infty(n)$ =: Number of steps on the Collatz sequence of n until $T^{(k)}(n) = 1$.

Divergent trajectories conjecture

The function $T : \mathbb{Z} \to \mathbb{Z}$ has no divergent trajectories, i.e., there exists no integer n_0 for which,

$$\lim_{k\to\infty} \left|T^{(k)}(n_0)\right| = \infty.$$

2.1.2 3x + 1 Conjecture (Second form)

Every integer $n \geq 2$ has a finite stopping time.

2.2 Unsolved problems formulated from 3x + 1 problem[21]

Analysis of the $3x + 1$ problem has resulted in the formulation of a large set of "easier" problems. At first glance some of these seem approachable, but they also remain unsolved, and are apparently difiicult. As samples, these include:

1. (**Finite Cycles Conjecture**) Does the $3x + 1$ function have finitely many cycles (i.e. finitely many purely periodic orbits on the integers)? This is conjectured to be the case.

2. (**Divergent Trajectories Conjecture-1**) Does the $3x+1$ function have a divergent trajectory, i.e., an integer starting value whose iterates are unbounded? This is conjectured not to be the case.

3. (**Divergent Trajectories Conjecture-2**) Does the $5m + 1$ function have a divergent trajectory? This is conjectured to be the case.

4. (**Infinite Permutations-Periodic Orbits Conjecture**) If a generalized Collatz function permutes the integers and is not globally of finite order, is it true that it has only finitely many periodic orbits? The original Collatz function $U(n)$, which is a permutation, was long ago conjectured to have finitely many cycles. A conjecture of this kind, imposing extra conditions an the permutation, was formulated by Venturini [39] [p. 303 top].

5. (**Infinite Permutations-Zero Density Conjecture**) If a generalized Collatz function permutes the integers, is it true that every orbit has a (natural) density? Under some extra hypotheses one may conjecture that all such orbits have density zero; compare Venturini [39] [Sec. 6].

.

Chapter 3

Materials and Methods

3.1 The Collatz function and the conjecture

The Collatz function is defined for any positive integer n,

$$T(n) = \begin{cases} \frac{n}{2}, & \text{If n is even} \\ 3n+1, & \text{If n is odd} \end{cases}$$

We use this function to generate sequence starting from some positive integer n. The next term of the sequence is obtained by applying the function to the previous term. The Collatz conjecture statement states that no matter what number you start with, it will eventually reach the number 1.

After the sequence reaches to number 1, there will be a loop of numbers in the sequence in the end such that $\{..., 1, 4, 2, 1, 4, 2, 1, ...\}$. The conjecture suggests that this is the only loop of numbers we can find for any sequence starting from any positive integer.

We can modify the Collatz function for some odd number k and positive integer m,

$$T(n) = \begin{cases} k \cdot 2^{m-1}, & \text{If } n = k \cdot 2^m \\ 3k+1, & \text{If } n = k \end{cases}$$

First, we can observe that any number is connected to some other number, Hence, there is a structure. We approach the Collatz conjecture in a constructive way such that we get a connected and divergent number system starting with number 1.

3.2 Connected Sub-Sequences

We Construct a sequence starting with an odd number k and all the other elements are even such that the nth term of the sequence as $k \cdot 2^n$,

$$\{\ k \cdot 2^n\}_{n=0}^{\infty}$$

Since the Collatz function connects every odd number k' with a unique mapping $3k' + 1$ to another even number $k \cdot 2^n$. We define $\{k' \cdot 2^n\}$ as a sub-sequence of a main-sequence $\{k \cdot 2^n\}$. Then,

$$k \cdot 2^n = 1 + 3k'$$

$$k \cdot 2^n \equiv 1 \pmod 3$$

Thus, for every $k'(\neq 1)$ number in a sub-sequence, there exist an odd number k in a main-sequence such that $k \neq k'$. Here, $k = k'$ if and only if $k' = 1$. Then we check the converse, what odd number k in a main-sequence has sub-sequences starting with odd numbers k'. We consider three cases,

- Case 1: If $k \equiv 0 \pmod 3$

  $\implies 0 \cdot 2^n \not\equiv 1 \pmod 3$, Since no k value satisfies the congruent, there are no sub-sequences for $n = 1, 2, 3, \ldots$

- Case 2: If $k \equiv 1 \pmod 3$

  $\implies 1 \cdot 2^n \equiv 1 \pmod 3$, Since the congruent is satisfied when n is an even number. There are sub-sequences when the number in the main-sequence is $k \cdot 2^{2n} \in \{k \cdot 2^n\}$ for n=1,2,3,...

- Case 3: If $k \equiv 2 \pmod 3$

  $\implies 2 \cdot 2^n \equiv 1 \pmod 3$, Since the congruent is satisfied when n is odd. There are sub-sequences when the number in the main-sequence is $k \cdot 2^{2n-1} \in \{k \cdot 2^n\}$ for n=1,2,3,...

Consequently k' odd elements in sub-sequence connects to $k \cdot 2^n$ elements in main-sequence. We call this as the Collatz structure.

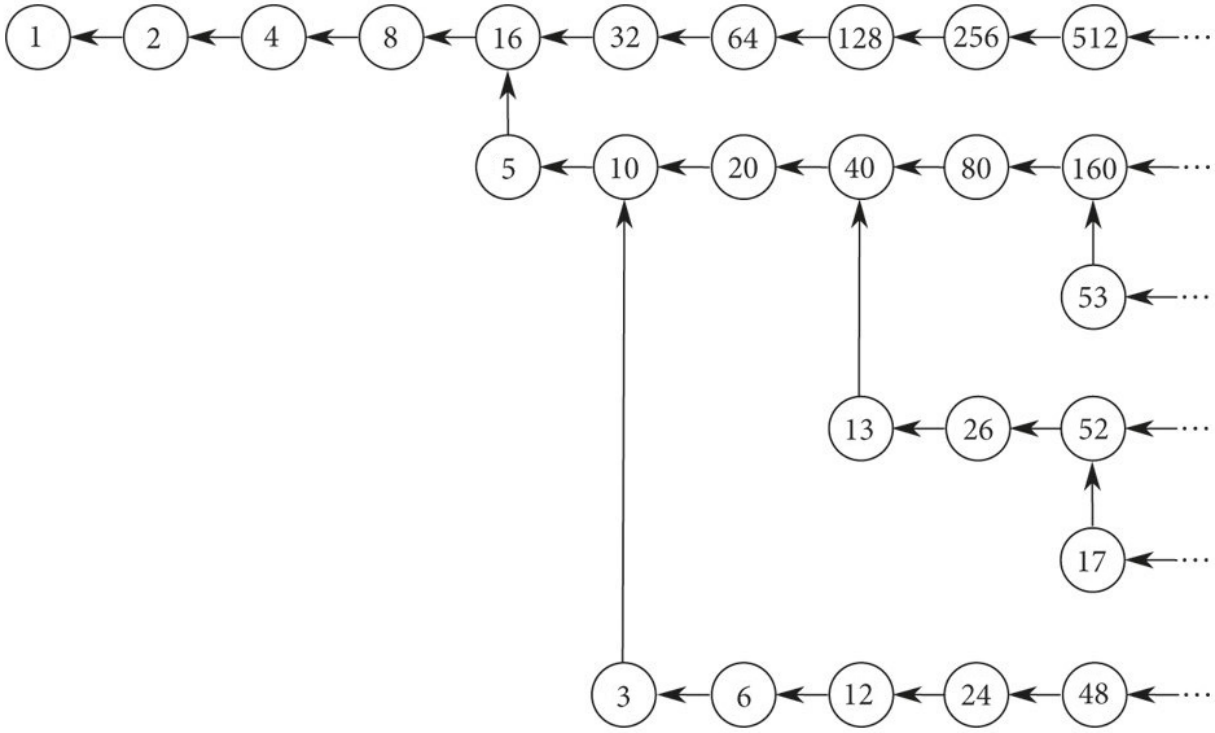


Figure 3.1: Visual diagram of connected sub-sequences(The Collatz structure)

We can define a function f from n^{th} level to $n-1^{th}$ level, mapping consecutive odd numbers k and k' in some generated Collatz sequence as, $f(k') = k$. As an example, $f(3) = 5$. We call this the odd Collatz function.

3.3 Sequences connected to $\{1 \cdot 2^m\}$ sequence

Suppose $\{1 \cdot 2^m\}$ is the 1st Level sequence and a pth Level sub-sequence be $\{k_p \cdot 2^m\}$.

1st Level $\implies \{1, 2, 4, 8, 16, ...\}$

2nd Level $\implies \{5 \cdot 2^m\}$, $\{21 \cdot 2^m\}$,$\{85 \cdot 2^m\}$,...

.

.

.

$(p-1)^{th}$ Level $\implies \{k_{p-1} \cdot 2^m\}$

p^{th} Level $\implies \{k_p \cdot 2^m\}$

We use the principle of strong induction at the nth level to prove that any number starting from any level will reach 1 eventually. Since all are even numbers (except 1) in the sequence of level 1 and by dividing an element $1 \cdot 2^m$(for some positive integer m), m times by 2, we get 1. Every element in 1st level will eventually reach to 1. Thus, it is true for $n = 1^{st}$ level. As the induction hypothesis, we assume that it is true for $n^{th}(< p^{th})$ level and let $n = p^{th}$ level contains sub-sequences of a main sequence at the $(p-1)^{th}$ Level. Thus, there exists an odd number k_{p-1} such that $k_p \cdot 2^n = 1 + 3k_{p-1}$. This implies that it is true for the $n = p^{th}$ level. The assertion still holds even if there are no sub-sequences at $n = p^{th}$ level, by the induction hypothesis. We can conclude that by the principle of strong induction, the assertion is true for all levels.

3.4 Odd number sequences

We already know that, there exist a sub-sequence for any odd number k in the main sequence,

- If $k \equiv 1 \pmod 3$, then the sub-sequences are generated when $k \cdot 2^{2n} \in \{k \cdot 2^n\}$ for $n = 1, 2, 3, ...$
- If $k \equiv 2 \pmod 3$, then the sub-sequences are generated when $k \cdot 2^{2n-1} \in \{k \cdot 2^n\}$ for $n = 1, 2, 3, ...$

We can define a function g from k to the first sub-sequence starting odd number k'_1,

$$k'_1 = g(k) = \begin{cases} \dfrac{k \cdot 2^2 - 1}{3}, & \text{If } k \equiv 1 \pmod 3 \\ \dfrac{k \cdot 2 - 1}{3}, & \text{If } k \equiv 2 \pmod 3 \end{cases}$$

So, we can define an odd number sequence generated from each odd number k, which is the sequence starting with k'_1,

$\left\{k'_1 \cdot 4^{n-1} + \sum_{i=0}^{n-2} 4^i\right\}_{n=1}^{\infty}$ and $4k'_n + 1 = k'_{n+1}$ for $n = 1, 2, 3, ...$

Since it is connected to k by the Collatz function, we can denote it as,

$[k] = \left\{k'_1 \cdot 4^{n-1} + \sum_{i=0}^{n-2} 4^i\right\}_{n=1}^{\infty}$.

We know that every even number of the form $k \cdot 2^m \in \{k \cdot 2^m\}$ for $m = 1, 2, 3, ...$ will converge to the starting odd number k. To prove the conjecture is true for all positive integers, we only need to show that, all sequences generated for all odd positive integers converge to 1, through the Collatz structure. We can modify collatz function using these odd number sequences for any odd number k and for some positive integer $m(\geq 2)$,

$$T(k) = \begin{cases} k'_1, & \text{If } k = k'_1 \cdot 4^{m-1} + \sum_{i=0}^{m-2} 4^i \\ f(k'_1), & \text{Else } k = k'_1 \end{cases}$$

We generate new sequence starting with g(k) for each k element in previous level odd sequence. We start from [k=1] and since $k \equiv 1 \pmod 3$ we get $1 = g(1)$. Here $k = g(k)$ only for k=1.

1st Generation $\implies [1] = \{1, 5, 21, 85, 341, ...\}$

2nd Generation $\implies [1], [5] = \{3, 13, ...\}, [85] = \{113, 453, ...\}, [341], ...$

3rd Generation $\implies |[1], [5], [85], [341], ...|[13] = \{17, 69, ...\}, ...|[113], ...|$

.

.

.

We can identify that 1st generation of odd numbers contain all the odd numbers of first two levels of collatz structure and n^{th} generation contains all odd numbers of first $n+1$ levels of the Collatz structure. Suppose, we have two similar odd k numbers in some generation, but $k = g(k)$ only for k=1 and the function g is one-to-one, then there should be two similar odd k numbers in previous generation also, which is a contradiction. We always get numbers different to previous all levels in the new level in the Collatz structure, except for the number 1. Most importantly, we would not have two similar odd numbers on two levels of the Collatz structure. Thus, the nth generation contains all odd numbers of first $n+1$ levels of the Collatz structure and each generation contains union of disjoint odd sequences.
Consider $[1] = \{1, 5, 21, 85, 341, ...\}$, if we generate new sequences using elements except 1, of this sequence, we get exact odd numbers similar to the collatz structure, we constructed previously. Thus, n^{th} generation contains same odd numbers as $n + 1^{th}$ level.

Since, the Collatz sequence is conjectured for any odd number, we can conjecture that, if we let the nth generation to the infinity, then we get all the odd positive integers as a disjoint union of sequences.

3.5 Infinite loops of sequences and the inverse Collatz function

3.5.1 Infinite loops of sequences

We define the process between two consecutive odd numbers in the Collatz sequence, as an iteration. The Collatz structure we created is opposite direction to collatz sequence and for one iteration in the Collatz sequence, for some odd number in n^{th} level reaches the $n - 1^{th}$ level. Suppose there is a an odd number k_0 such that after repeating the process, it never comes to the 1st level containing the sequence, $\{1 \cdot 2^n\}$. Then there exist a parallel system which is not connected the Collatz structure, we constructed. Also in that system, there exists an infinite loop of sequences such that there are two similar odd k_0 values after q iterations($k_q = k_0$),

$$
\begin{aligned}
k_0 \cdot 2^{n_0} &= 1 + 3k_{q-1} &&= T(k_{q-1}) && q^{th} \text{ iteration} \\
&\cdot &&\cdot && \cdot \\
&\cdot &&\cdot && \cdot \\
&\cdot &&\cdot && \cdot \\
k_2 \cdot 2^{n_2} &= 1 + 3k_1 &&= T(k_1) && 2^{nd} \text{ iteration} \\
k_1 \cdot 2^{n_1} &= 1 + 3k_0 &&= T(k_0) && 1^{st} \text{ iteration}
\end{aligned}
$$

We know that this loop of numbers contained in the system generated by odd number k_0 in the sequence $\{k_0 \cdot 2^n\}$. Without loss of generality, let $k_0 = max\{k_0, k_1, ..., k_{q-1}\}$.

3.5.2 The inverse Collatz function

We know that there are infinitely many ways that odd numbers converge to 1, through the Collatz structure and there is a leftmost path for some particular odd number k.

Since, when $k \equiv 0 \pmod 3$ the Collatz structure doesn't have sub-sequences, we map those k's to $k \cdot 2^2 + 1$.

We can modify function g for all odd numbers,

$$
g(k) = \begin{cases} \dfrac{k \cdot 2^2 - 1}{3}, & \text{If } k \equiv 1 \pmod 3 \\ \dfrac{k \cdot 2 - 1}{3}, & \text{If } k \equiv 2 \pmod 3 \\ k \cdot 2^2 + 1, & \text{If } k \equiv 0 \pmod 3 \end{cases}
$$

Since $k = g(k)$ only for k=1, we have a convergent sequence $\{1, 1, 1, 1, ...\}$ for $k = 1$. Thus, for any odd number k not equal to 1, we can conjecture that the sequence generated from this function is divergent. We can observe that,

for odd numbers of the form $k \equiv 1 \pmod 3$ and $k \equiv 0 \pmod 3$, we get $k < g(k)$.

for odd numbers of the form $k \equiv 2 \pmod 3$, we get $k > g(k)$. Here $k = 3m + 2$ for some positive odd integer m. Since k is an odd number, m should be an odd number. Let $m = 2r - 1$ for some positive integer $r = 1, 2, 3, ...$

Then, $k = 6r - 1 \equiv 2 \pmod 3$ for some positive integer $r = 1, 2, 3, ...$

3.5.3 The sequences generated by the function g are divergent for all positive odd integers

We use the principle of strong induction on r to prove that,
the sequences generated for all odd numbers $k \leq 6r-1$, we have an odd number $g^t(k) > 6r-1$ for some integer t.

First, we take $r = 1$. Then we get $6r - 1 = 5$ and the sequence generated for $k = 3$ is $\{13, 17, 11, 7, 9, 37...\}$ and the sequence generated for $k = 5$ is $\{3, 13, 17, 11, 7, 9, 37...\}$. Since we have $g^2(k) > 5$ for all $k \leq 5$, the assertion is true for $r = 1$.
Then, we take $r = 2, 3, 4$, so we get $6r - 1 = 11, 17, 23$. Since we have $g^t(k) > 23$ for some t for the sequences generated for all $k \leq 23$, the assertion is true for $r = 1, 2, 3, 4$.
As the induction hypothesis, assume the assertion is true for $4 < r < n$, then for all $k \leq 6n-7$, we have odd numbers $g^t(k) > 6n - 7$ for some integer t. After that, we consider $r = n$. We want to show that for all $k \leq 6n-1$, there exist an odd number $g^t(k) > 6n-1$ for some integer t.

- If $k = 6n - 5 \equiv 1 \pmod 3$, then we get $g(6n-5) = \dfrac{(6n-5)\cdot 2^2 - 1}{3} = \dfrac{24n-21}{3} = 8n - 7 > 6n - 1$ for $n > 4$.

- If $k = 6n-3 \equiv 0 \pmod 3$, then we get $g(6n-3) = (6n-3)\cdot 2^2 + 1 = 24n - 11 > 6n - 1$ for $n > 4$.

- If $k = 6n - 1 \equiv 2 \pmod 3$, then we get $g(6n-1) = \dfrac{(6n-1)\cdot 2 - 1}{3} = \dfrac{12n-3}{3} = 4n-1 < 6n-1$. We want to show that $g^t(k) > 6n-1$ for some t, but this case is not true if and only if $g^t(6n-1) = 6n-1$ for some t and there is a loop in the sequence generated for $6n-1$ such that $k_0 = 6n-1 = max\{k_0, k_1, ..., k_{q-1}\}$. Also after q(finite) number of iterations, we get $g^q(6n-1) = 6n-1$. In the section 3.4, we found that, there are no similar odd numbers in two levels in the Collatz structure. Hence, $g^t(6n-1) \neq 6n-1$ for any t. The assertion is true for $r = n$. By the principle of strong induction the assertion is true for all n.

Since we chose $k = 6r-1$ arbitrarily for any odd number $k \equiv 2 \pmod 3$, the sequence generated by function g is divergent. We can conclude that the sequences generated from the function is divergent for all odd numbers such that the Collatz structure contains all the positive odd integers. We can conclude that, there can not exist a loop of odd numbers in a separate structure, which is not connected the Collatz structure. Thus, the Collatz conjecture is true for any positive integer.

An unsuccessful approach on infinite loops of sequences

Suppose we generate a sequence using the Collatz function starting with an odd number k_0 and after q iterations we reach k_q. But we should get a different odd number k, after q iterations such that $k_q \neq k_0$, we can conjecture that for some positive integer q,

If $\sum_{i=1}^{q} n_i > 2q$, then $k_q < k_0$.
If $\sum_{i=1}^{q} n_i < 2q$, then $k_q > k_0$.
If $\sum_{i=1}^{q} n_i = 2q$, then $k_q \neq k_0$.

We use the principle of mathematical induction on the qth iteration to prove that we get different values for k_q, after q iterations, except for $k_q = 1$.

- First we consider two cases $q = 1$ and $q = 2$.
  - For $q = 1$, we have $k_1 = \dfrac{1+3k_0}{2^{n_1}} = \dfrac{f(k_0)}{2^{n_1}}$ and $n_i \in \{1, 2, 3, ...\}$
    If $\sum_{i=1}^{q} n_i > 2$, then we get $n_1 > 2$ which implies $k_1 < k_0$.
    If $\sum_{i=1}^{q} n_i < 2$, then we get $n_1 < 2$ which implies $k_1 > k_0$.
    If $\sum_{i=1}^{q} n_i = 2$, then we get $n_1 = 2$ which implies $k_1 \leq k_0$.
    True for $q = 1$.
  - For $q = 2$, we have $k_2 = \dfrac{1+3\left(\dfrac{1+3k_0}{2^{n_1}}\right)}{2^{n_2}} = \dfrac{f(k_1)}{2^{n_2}}$ and $n_i \in \{1, 2, 3, ...\}$

    we have 3 sub cases for this case.

1. For $n_1 > 2$, we have $k_1 < k_0$

   If $\sum_{i=1}^{q} n_i > 4$ and $n_2 \geq 1$, then we get $k_2 < k_0$.

   If $\sum_{i=1}^{q} n_i < 4$ and $n_2 = 1$, then we get $k_2 < k_0$.

   If $\sum_{i=1}^{q} n_i = 4$ and $n_2 = 1$, then we get $k_2 \neq k_0$.

2. For $n_1 < 2$, we have $n_1 = 1 \implies k_1 > k_0$

   If $\sum_{i=1}^{q} n_i > 4$ and $n_2 > 3$, then we get $k_2 < k_0$.

   If $\sum_{i=1}^{q} n_i < 4$ and $n_2 < 3$, then we get $k_2 > k_0$.

   If $\sum_{i=1}^{q} n_i = 4$ and $n_2 = 3$, then we get $k_2 \neq k_0$.

3. For $n_1 = 2$, we have $k_1 \neq k_0$

   If $\sum_{i=1}^{q} n_i > 4$ and $n_2 > 2$, then we get $k_2 < k_0$.

   If $\sum_{i=1}^{q} n_i < 4$ and $n_2 = 1$, then we get $k_2 > k_0$.

   If $\sum_{i=1}^{q} n_i = 4$ and $n_2 = 2$, then we get $k_2 \neq k_0$.

   True for $q = 2$.

- As the induction hypothesis, we assume the assertion is true for $q = p$,

  $k_p = \dfrac{1 + 3k_{p-1}}{2^{n_p}} = \dfrac{f(k_{p-1})}{2^{n_p}}$ and $n_i \in \{1, 2, 3, ...\}$

  For $\sum_{i=1}^{p} n_i > 2p$, we have $k_p < k_0$.

  For $\sum_{i=1}^{p} n_i < 2p$, we have $k_p > k_0$.

  For $\sum_{i=1}^{p} n_i = 2p$, we have $k_p \neq k_0$.

- Then we consider when $q = p + 1$,

  $k_{p+1} = \dfrac{1 + 3(\dfrac{1 + 3k_{p-1}}{2^{n_p}})}{2^{n_{p+1}}} = \dfrac{f(k_p)}{2^{n_{p+1}}}$ and $n_i \in \{1, 2, 3, ...\}$

  we have 3 sub cases for this case.

  1. For $\sum_{i=1}^{p} n_i > 2p$, we have $k_p < k_0$

     If $\sum_{i=1}^{p+1} n_i > 2(p + 1)$ and $n_{p+1} > 2$, then we get $k_{p+1} < k_0$.

     If $\sum_{i=1}^{p+1} n_i < 2(p + 1)$ and $n_{p+1} = 1$, then we get $k_{p+1} < k_0$.

If $\sum_{i=1}^{p+1} n_i = 2(p+1)$ and $n_{p+1} = 1$, then we get $k_{p+1} \neq k_0$.

2. For $p \leq \sum_{i=1}^{p} n_i < 2p$, we have $k_p > k_0$

   If $\sum_{i=1}^{p+1} n_i > 2(p+1)$ and $n_{p+1} > p+2$, then we get $k_{p+1} < k_0$.

   If $\sum_{i=1}^{p+1} n_i < 2(p+1)$ and $n_{p+1} < p+2$, then we get $k_{p+1} > k_0$.

   If $\sum_{i=1}^{p+1} n_i = 2(p+1)$ and $n_{p+1} \leq p+2$, then we get $k_{p+1} \neq k_0$.

3. For $\sum_{i=1}^{p} n_i = 2p$, we have $k_p \neq k_0$

   If $\sum_{i=1}^{p+1} n_i > 2(p+1)$ and $n_{p+1} > 2$, then we get $k_{p+1} < k_0$.

   If $\sum_{i=1}^{p+1} n_i < 2(p+1)$ and $n_{p+1} = 1$, then we get $k_{p+1} \geq k_0$.

   If $\sum_{i=1}^{p+1} n_i = 2(p+1)$ and $n_{p+1} = 2$, then we get $k_{p+1} \neq k_0$.

By the Principle of Mathematical Induction, the assertion is true for all $q(k_q \neq k_0)$ when $\sum_{i=1}^{p} n_i \neq 2p$. But, for the case $\sum_{i=1}^{p} n_i = 2p$ such that $k_p \neq k_0(k_p < k_0)$, $\sum_{i=1}^{p+1} n_i < 2(p+1)$ and $n_{p+1} = 1$, there can be a loop which is $k_{p+1} = k_0$. When $q = 1$, we find the first loop after first iteration, which is [**1,4,2,1,4,2,1,...**].

3.6 Odd numbers of the form 6n − 1 and 6n + 1

The Fundamental Theorem of Arithmetic states that every natural number greater than 1, can be written as a product of distinct prime numbers. For some positive integer $n(\geq 1)$,

$6n - 1 = 2^{\alpha_1}3^{\alpha_2}5^{\alpha_3}P_4^{\alpha_4}...P_j^{\alpha_j} - 1$ for $\alpha_j, j \in \mathbb{N}$ and $P_j \in$ Primes

$6n - 1 = 2^l 3^m q - 1$; for some positive integers l,m and odd number q.

$6n + 1 = 2^l 3^m 5^{\alpha_3}P_4^{\alpha_4}...P_j^{\alpha_j} + 1$ for $\alpha_j, j \in \mathbb{N}$ and $P_j \in$ Primes

$6n + 1 = 2^l 3^m q + 1$; for some positive integers l,m and odd number q.

We have already defined the function g(inverse odd Collatz function). We can apply g only if $l, m \neq 0$ and we know that, $k = 6n - 1 \equiv 2 \pmod 3$ and $k = 6n + 1 \equiv 1 \pmod 3$.

$$g(k) = \begin{cases} \dfrac{k \cdot 2^2 - 1}{3}, & \text{If } k \equiv 1 \pmod 3 \\ \dfrac{k \cdot 2 - 1}{3}, & \text{If } k \equiv 2 \pmod 3 \\ k \cdot 2^2 + 1, & \text{If } k \equiv 0 \pmod 3 \end{cases}$$

We have defined the function f in the section 3.2, for two consecutive odd numbers(k' and k) in the Collatz sequence.

$$f(k') = k$$

3.6.1 Odd numbers of the form $\mathbf{6n+1 = 2^l 3^m q + 1}$

First we apply g once,
$g(2^l 3^m q + 1) = \dfrac{(2^l 3^m q + 1) \cdot 2^2 - 1}{3} = 2^{l+2} 3^{m-1} q + 1 \equiv 1 \pmod 3$
Here we lose one factor of 3 and gain 2 factors of 2 from $2^l 3^m q$, but still $2^{l+2} 3^{m-1} q + 1 \equiv 1 \pmod 3$

We can apply g, m times $\implies$
$g^m(2^l 3^m q + 1) = 2^{l+2m} q + 1 \not\equiv 1 \pmod 3$ and $f^m(2^{l+2m} q + 1) = 2^l 3^m q + 1$.

As long as $m \neq 0$, for some finite number m, the sequence generated by g increases m times. Also, we can go back using the function f.

3.6.2 Odd numbers of the form $\mathbf{6n-1 = 2^l 3^m q - 1}$

First we apply g once,
$g(2^l 3^m q - 1) = \dfrac{(2^l 3^m q - 1) \cdot 2 - 1}{3} = 2^{l+1} 3^{m-1} q - 1 \equiv 2 \pmod 3$
Here we lose one factor of 3 and gain one factor of 2 from $2^l 3^m q$, but still $2^{l+2} 3^{m-1} q - 1 \equiv 2 \pmod 3$

We can apply g, m times $\implies$
$g^m(2^l 3^m q - 1) = 2^{l+m} q - 1 \not\equiv 2 \pmod 3$ and $f^m(2^{l+m} q - 1) = 2^l 3^m q - 1$.

As long as $m \neq 0$, for some finite number m, the sequence generated by g decreases m times. Also, we can go back using the function f.

Chapter 4

Results

4.1 Similar Sequences to the Collatz sequence

The Fundamental Theorem of Arithmetic states that every natural number greater than 1, can be written as a product of distinct prime numbers. For some positive integer $n(\geq 1)$

$n = 2^{\alpha_1}3^{\alpha_2}5^{\alpha_3}P_4^{\alpha_4}...P_j^{\alpha_j}$ for $\alpha_j, j \in \mathbb{N}$ and $P_j \in$ Primes

$n = 2^{\alpha_1}3^{\alpha_2}...P^{\alpha_n}k$; for some odd number k

$$f(n) = \begin{cases} \frac{n}{2}, & \text{If } n = 2^{\alpha_1}3^{\alpha_2}5^{\alpha_3}k \\ \frac{n}{3}, & \text{If } n = 3^{\alpha_2}5^{\alpha_3}k \ ; \ \alpha_1 = 0 \\ . & . \\ . & . \\ . & . \\ Pn+1, & \text{If } n = k \ ; \ \alpha_1, \alpha_2, .., \alpha_n = 0 \end{cases}$$

Similarly we can define functions, $Pn + 1$, for any prime P. As an example for $P = 5$, we can define the function as,

$$f(n) = \begin{cases} \frac{n}{2}, & \text{If } n = 2^{\alpha_1}3^{\alpha_2}k \\ \frac{n}{3}, & \text{If } n = 3^{\alpha_2}k \ ; \ \alpha_1 = 0 \\ 5n+1, & \text{If } n = k \ ; \ \alpha_1, \alpha_2 = 0 \end{cases}$$

4.1.1 Pn + 1 Sequence lengths and loops

We use $Pn+1$ function to construct sequences for each prime $P(< 50)$. Python codes are used to construct sequences starting with 1-1,000,000. The code is written to calculate lengths of the sequences and numbers repeat in the sequences for first one million numbers. So we can determine the loops that exist on those systems and by generating sequences for those numbers we found, we can check whether they lie on the same sequence or not. We were able to conclude that those numbers which are repeating occurs in a single sequence as a loop(not connected to 1), for whatever the repeating number we generated the sequence with.

Following illustrations represent those $Pn+1$ Sequences' lengths and loops for each prime P. We denote the length and loops as,

- Length: number of steps it takes to reach the smallest number of a loop.
- Loop: Same number is repeated in the sequence.

3n + 1 Sequence lengths and loops

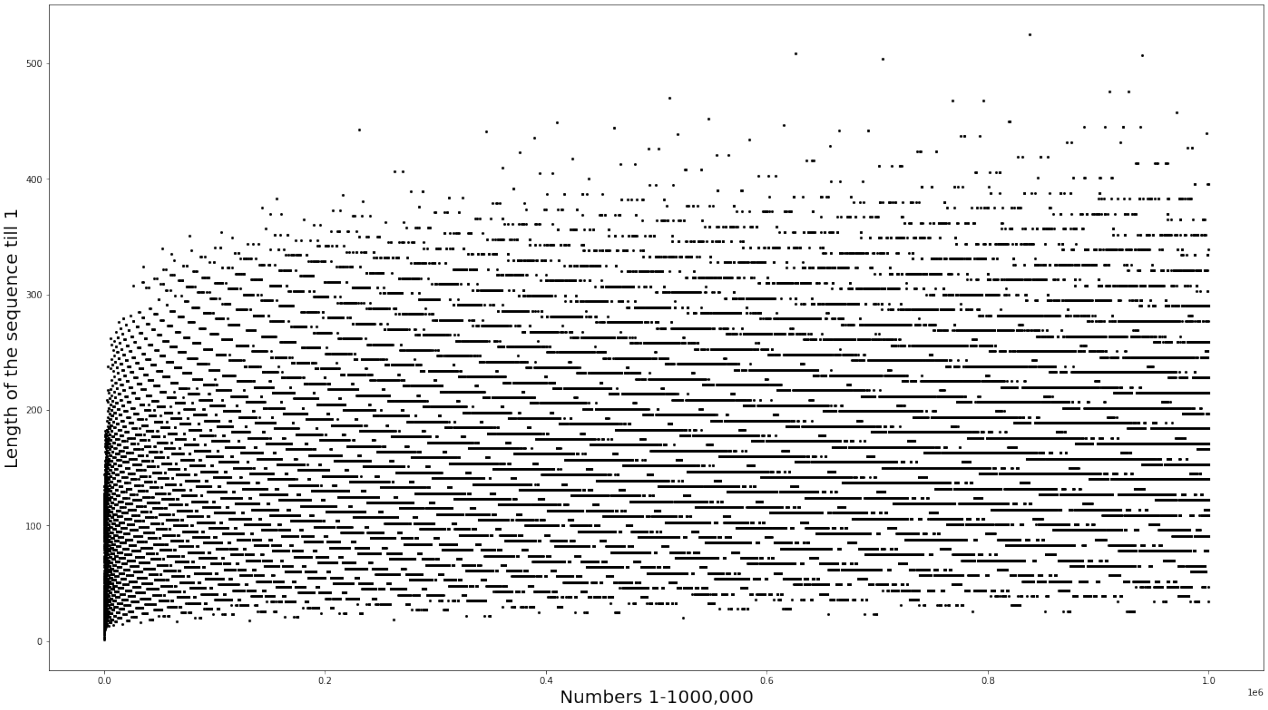


Figure 4.1: Number of terms up to 1, in the sequences generated by the $3n+1$ function using Python.

- Loop : $[1, 4, 2, 1, ..]$

5n + 1 Sequence lengths and loops

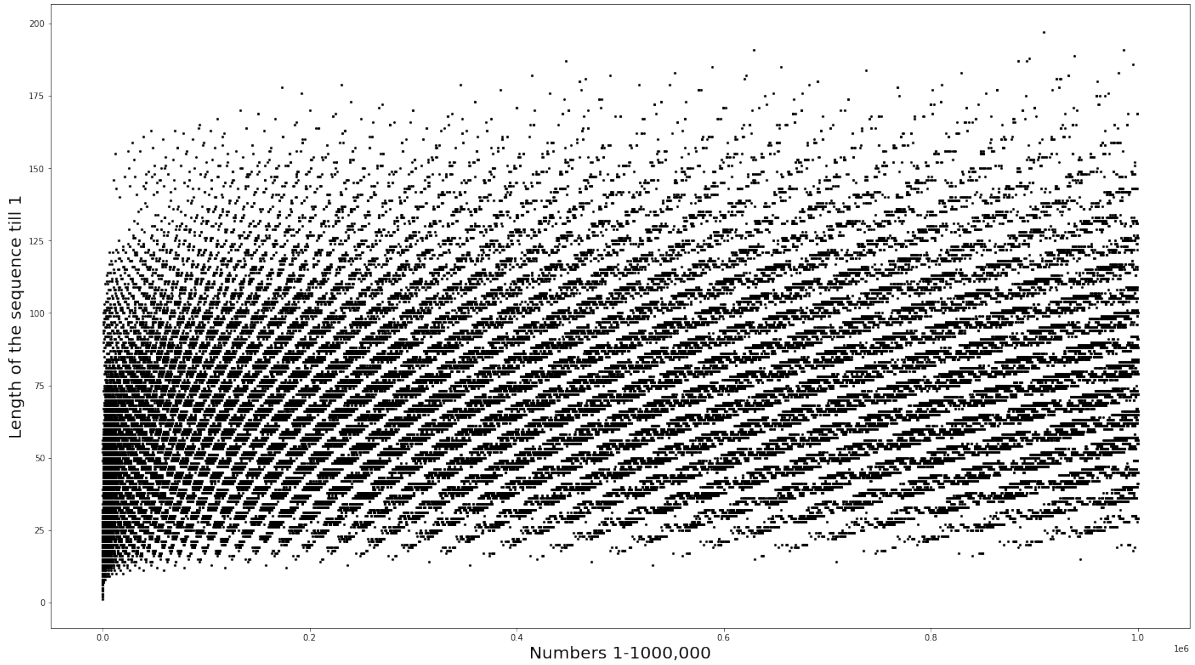


Figure 4.2: Number of terms up to 1, in the sequences generated by the $5n + 1$ function using Python.

- Loop : $[1, 6, 3, 1, ...]$

7n + 1 Sequence lengths and loops

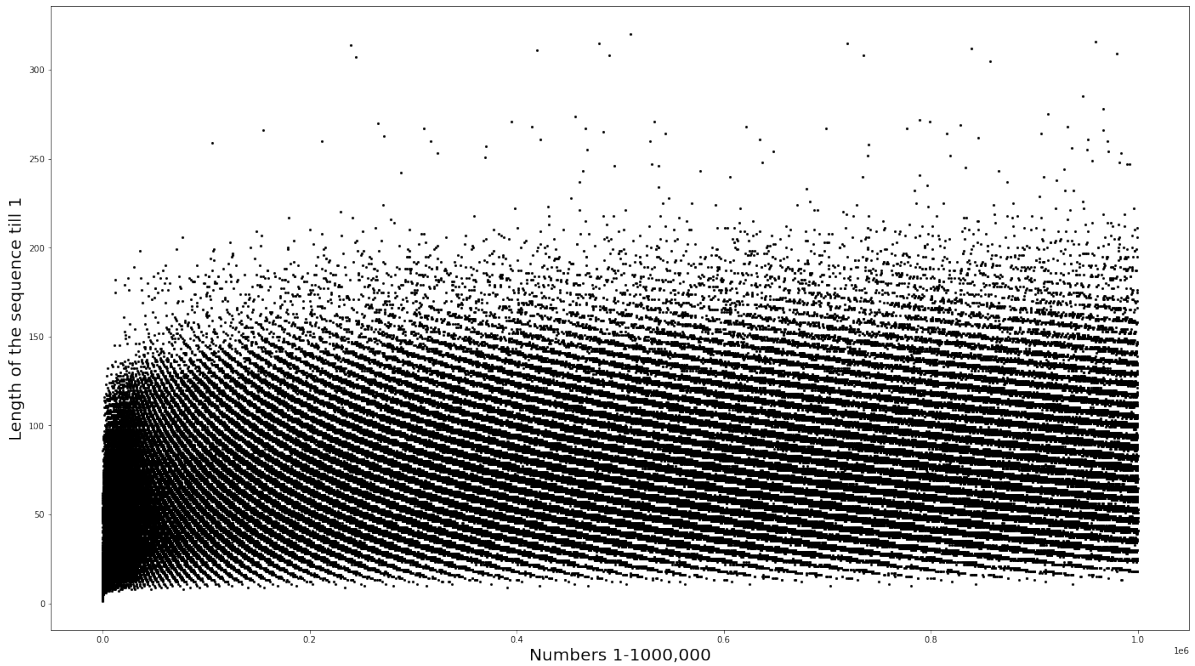


Figure 4.3: Number of terms up to 1, in the sequences generated by the $7n + 1$ function using Python.

- Loop : $[1, 8, 4, 2, 1, ...]$

11n + 1 Sequence Lengths and loops

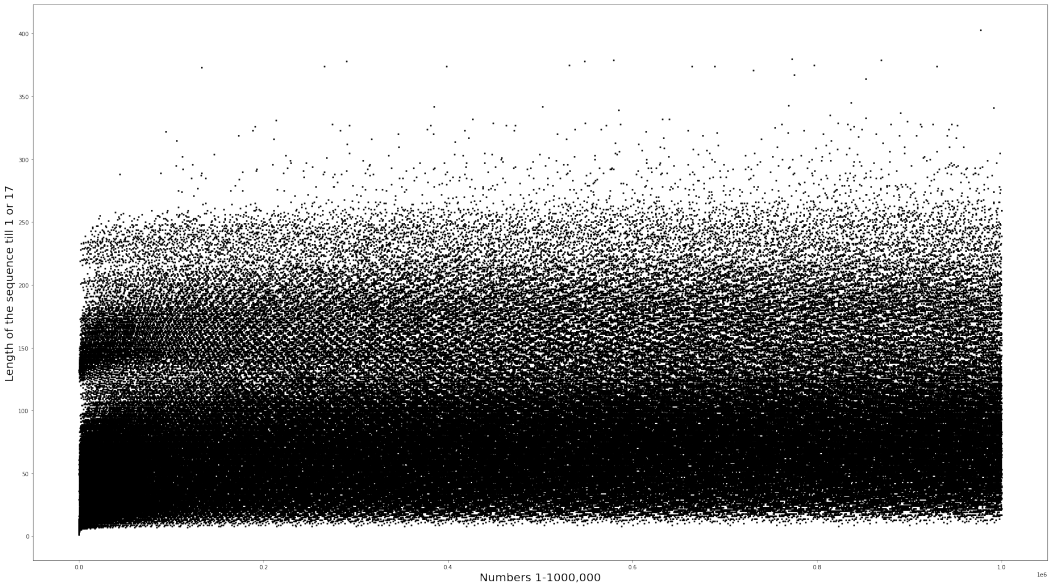


Figure 4.4: Number of terms up to a loop, in the sequences generated by the $11n + 1$ function using Python.

- Loop 1 : $[1, 12, 6, 3, 1, ...]$ and Loop 2 : $[17, 188, 94, 47, 518, 259, 37, 408, 204, 102, 51, 17, ...]$

13n + 1 Sequence lengths and loops

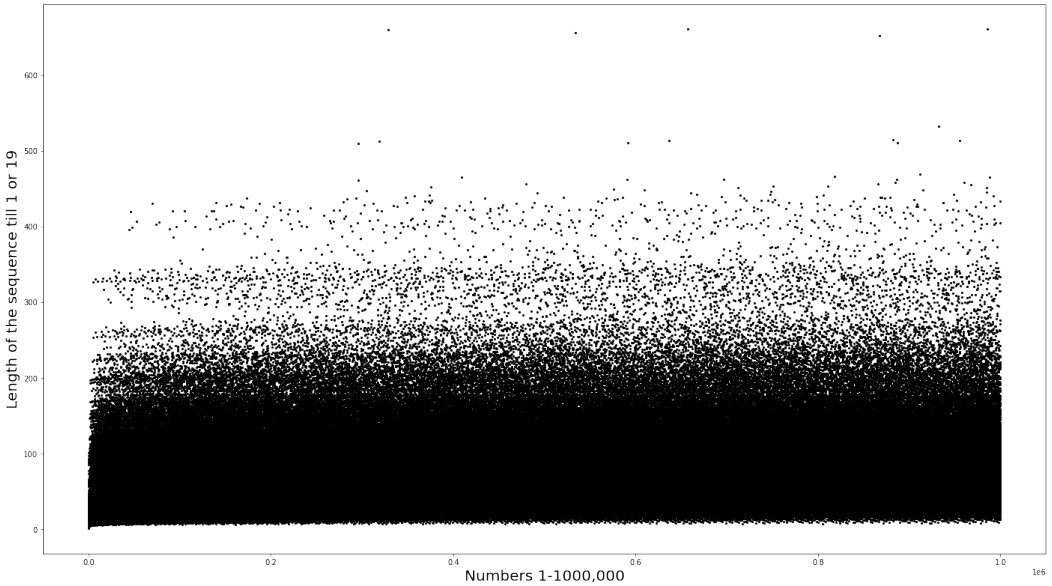


Figure 4.5: Number of terms up to a loop, in the sequences generated by the $13n + 1$ function using Python.

- Loop 1: $[1, 14, 7, 1, ...]$
- Loop 2: [19 , 248, 124, 62, 31, 404, 202, 101, 1314, 657, 219, 73, 950, 475, 95, 19, ...]

17n + 1 Sequence lengths and loops

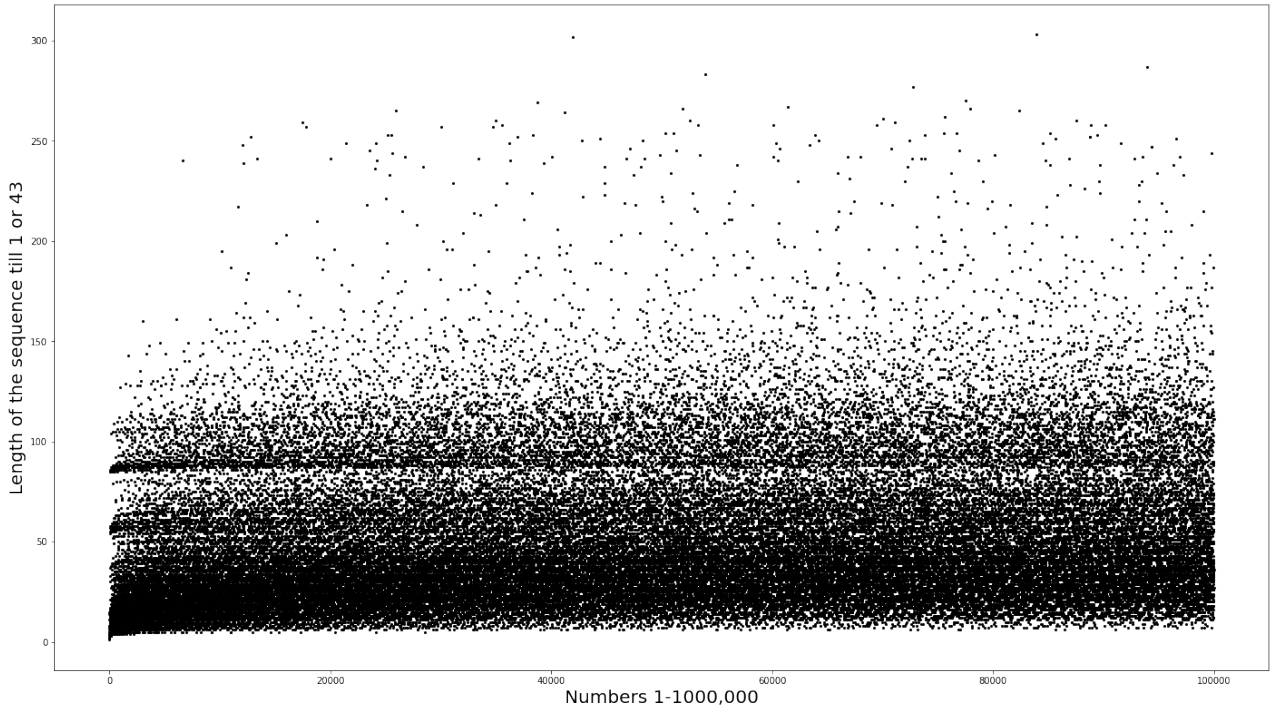


Figure 4.6: Number of terms up to a loop, in the sequences generated by the $17n + 1$ function using Python.

- Loop 1 : $[1, 18, 9, 3, 1, ...]$,
- Loop 2 : [43, 732, 366, 183, 61, 1038, 519, 173, 2942, 1471, 25008, 12504, 6252, 3126, 1563, 521, 8858, 4429, 75294, 37647, 12549, 4183, 71112, 35556, 17778, 8889, 2963, 50372, 25186, 12593, 1799, 257, 4370, 2185, 437, 7430, 3715, 743, 12632, 6316, 3158, 1579, 26844, 13422, 6711, 2237, 38030, 19015, 3803, 64652, 32326, 16163, 2309, 39254, 19627, 333660, 1668300, 83415, 27805, 5561, 94538, 47269, 803574, 401787, 133929, 44643, 14881, 252978, 126489, 42163, 3833, 65162, 32581, 553878, 276939, 92313, 30771, 10257, 3419, 263, 4472, 2236, 1118, 559, 43,...]

19n + 1 Sequence lengths

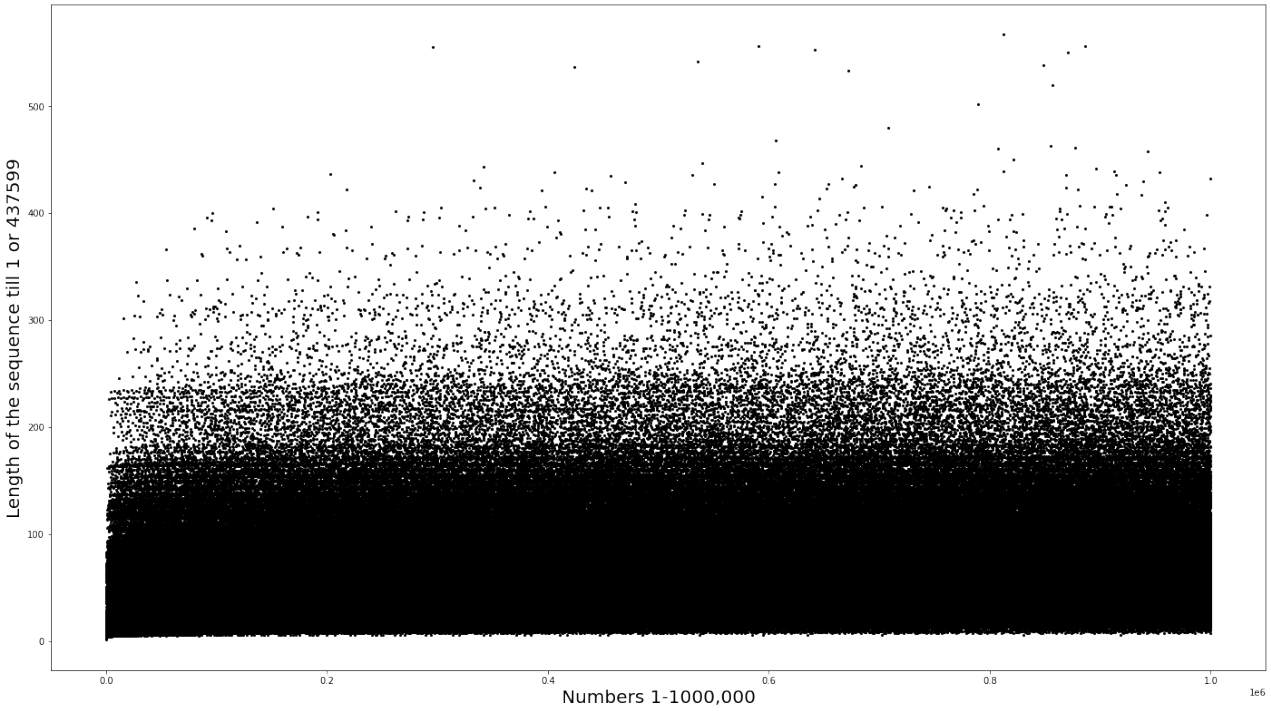


Figure 4.7: Number of terms up to a loop, in the sequences generated by the $19n + 1$ function using Python.

$19n + 1$ Sequence loops

- Loop 1 : [1, 20, 10, 5, 1, ...]
- Loop 2 : [437599, 8314382, 4157191, 78986630, 39493315, 7898663, 150074598, 75037299, 25012433, 475236228, 237618114, 118809057, 39603019, 752457362, 376228681, 7148344940, 3574172470, 1787086235, 357417247, 32492477, 617357064, 308678532, 154339266, 77169633, 25723211, 488741010, 244370505, 81456835, 16291367, 309535974, 154767987, 51589329, 17196443, 1563313, 29702948, 14851474, 7425737, 675067, 12826274, 6413137, 121849604, 60924802, 30462401, 578785620, 289392810, 144696405, 48232135, 9646427, 1378061, 26183160, 13091580, 6545790, 3272895, 1090965, 363655, 72731, 1381890, 690945, 230315, 46063, 875198, 437599, ...]

23n + 1 Sequence lengths and loops

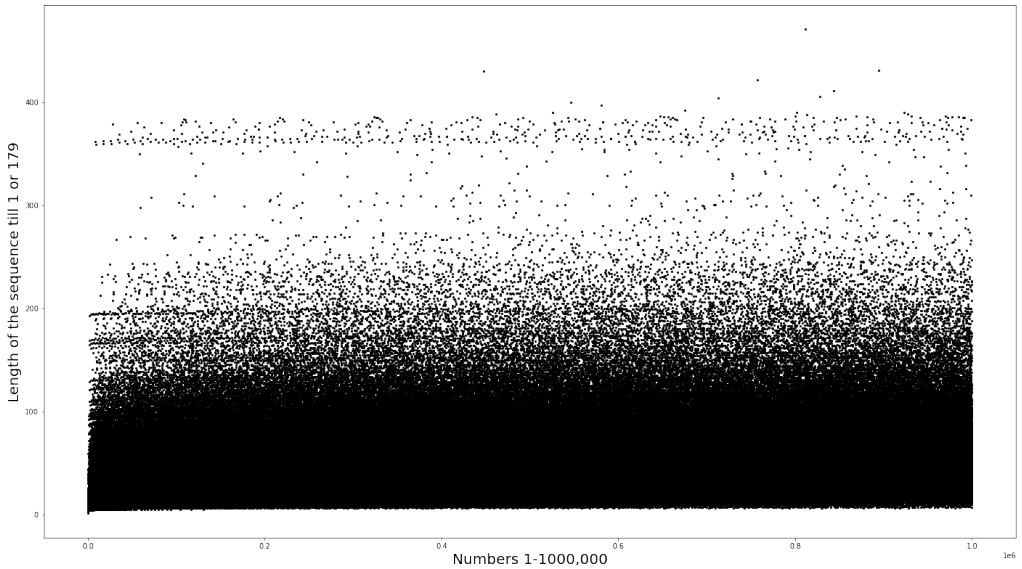


Figure 4.8: Number of terms up to a loop, in the sequences generated by the $23n + 1$ function using Python.

- Loop 1 : [1, 24, 12, 6, 3, 1, ...]
- loop 2 : [179, 4118, 2059, 47358, 23679, 7893, 2631, 877, 20172, 10086, 5043, 1681, 38664, 19332, 9666, 4833, 1611, 537, 179, ...]

29n + 1 Sequence lengths and loops

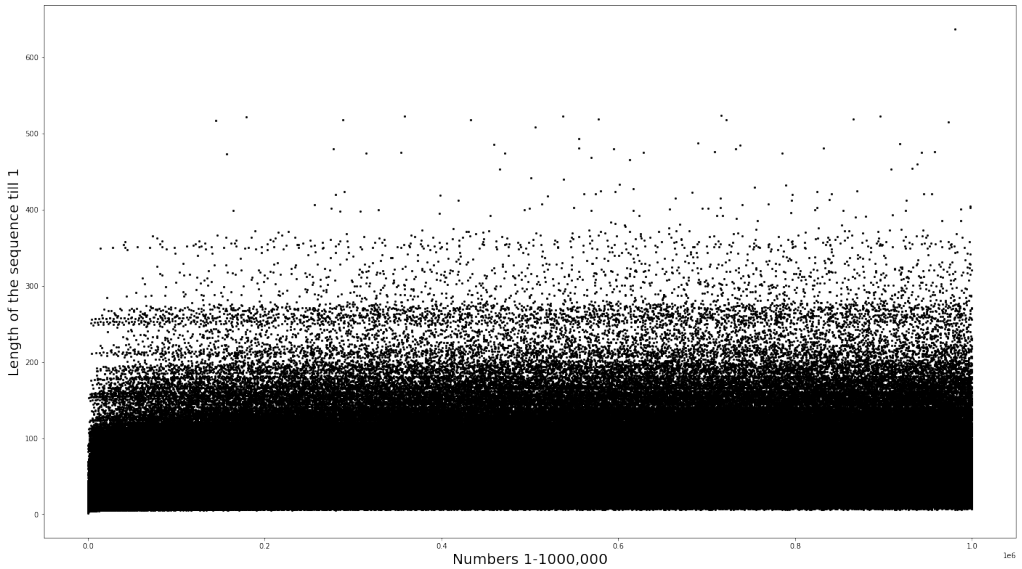


Figure 4.9: Number of terms up to 1, in the sequences generated by the $29n + 1$ function using Python.

- Loop : $[1, 30, 15, 5, 1, ...]$

31n + 1 Sequence Lengths

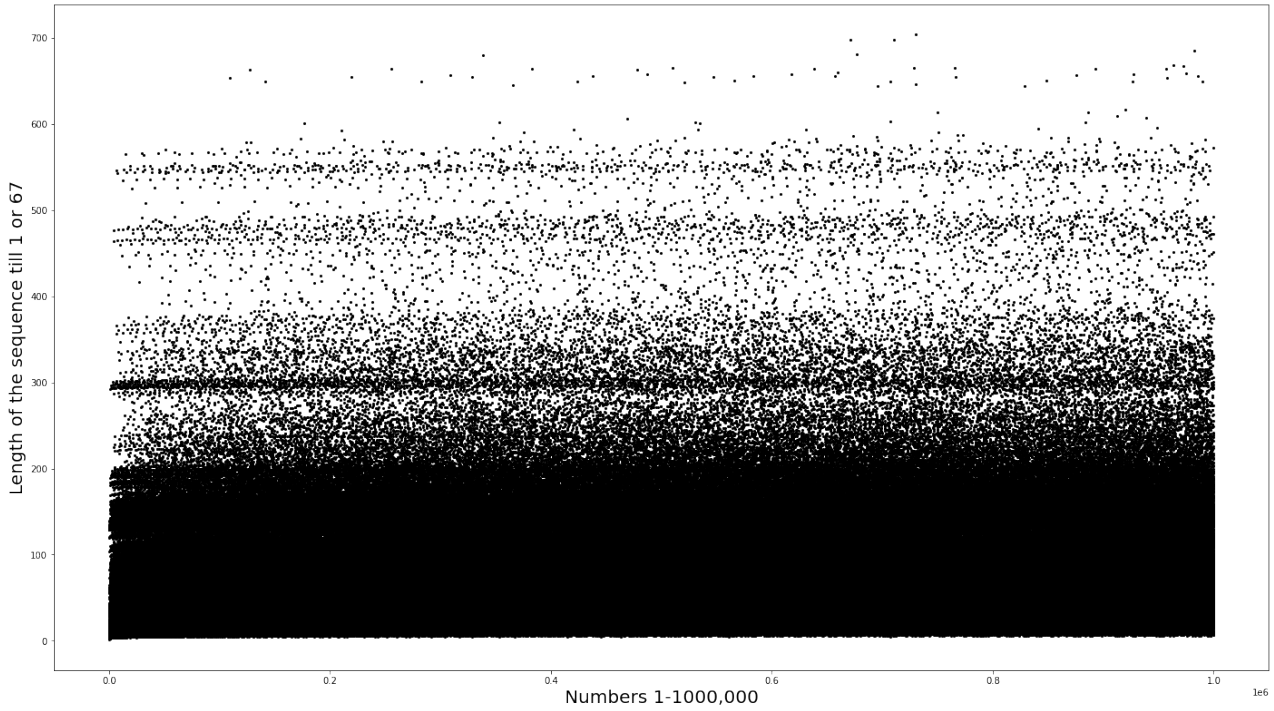


Figure 4.10: Number of terms up to a loop, in the sequences generated by the $31n+1$ function using Python.

31n + 1 Sequence loops

- Loop 1 : [1, 32, 16, 8, 4, 2, 1, ...]
- Loop 2 : [67, 2078, 1039, 32210, 16105, 3221, 99852, 49926, 24963, 8321, 257952, 128976, 64488, 32244, 16122, 8061, 2687, 83298, 41649, 13883, 430374, 215187, 71729, 10247, 317658, 158829, 52943, 4813, 149204, 74602, 37301, 3391, 105122, 52561, 1629392, 814696, 407348, 203674, 101837, 3156948, 1578474, 789237, 263079, 87693, 29231, 906162, 453081, 151027, 4681838, 2340919, 334417, 10366928, 5183464, 2591732, 1295866, 647933, 58903, 4531, 197, 6108, 3054, 1527, 509, 15780, 7890, 3945, 1315, 263, 8154, 4077, 1359, 453, 151, 4682, 2341, 72572, 36286, 18143, 562434, 281217, 93739, 2905910, 1452955, 290591, 41513, 1286904, 643452, 321726, 160863, 53621, 1849, 57320, 28660, 14330, 7165, 1433, 44424, 22212, 11106, 5553, 1851, 617, 19128, 9564, 4782, 2391, 797, 24708, 12354, 6177, 2059, 71, 2202, 1101, 367, 11378, 5689, 176360, 88180, 44090, 22045, 4409, 136680, 68340, 34170, 17085, 5695, 1139, 67, ...]

37n + 1 Sequence Lengths

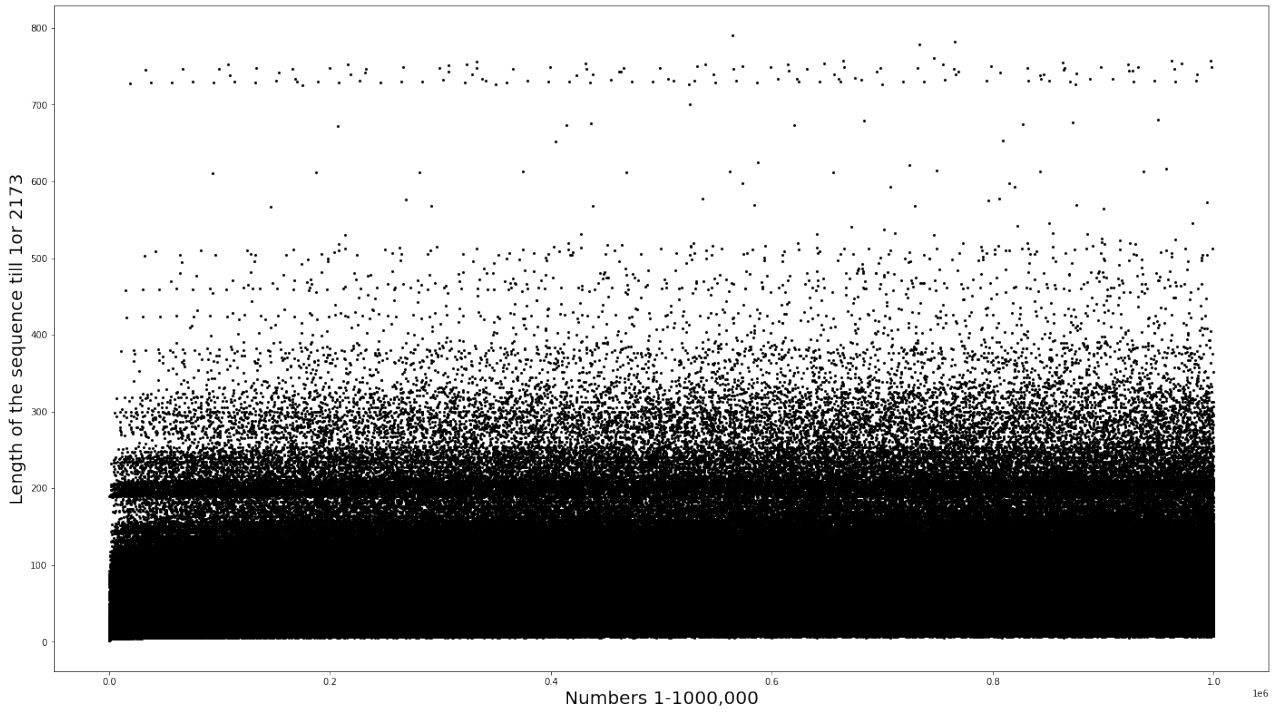


Figure 4.11: Number of terms up to a loop, in the sequences generated by the $37n + 1$ function using Python.

37n + 1 Sequence loops

- Loop 1 : [1, 38, 19, 1, ...]
- Loop 2 : [2173, 80402, 40201, 5743, 212492, 106246, 53123, 7589, 280794, 140397, 46799, 1731564, 865782, 432891, 144297, 48099, 16033, 593222, 296611, 42373, 1567802, 783901, 29004338, 14502169, 1318379, 48780024, 24390012, 12195006, 6097503, 2032501, 75202538, 37601269, 1391246954, 695623477, 30244499, 975629, 36098274, 18049137, 6016379, 222606024, 111303012, 55651506, 27825753, 9275251, 545603, 20187312, 10093656, 5046828, 2523414, 1261707, 420569, 15561054, 7780527, 2593509, 864503, 31986612, 15993306, 7996653, 2665551, 888517, 126931, 18133, 670922, 335461, 47923, 2819, 104304, 52152, 26076, 13038, 6519, 2173, ...]

41n + 1 Sequence Lengths and loops

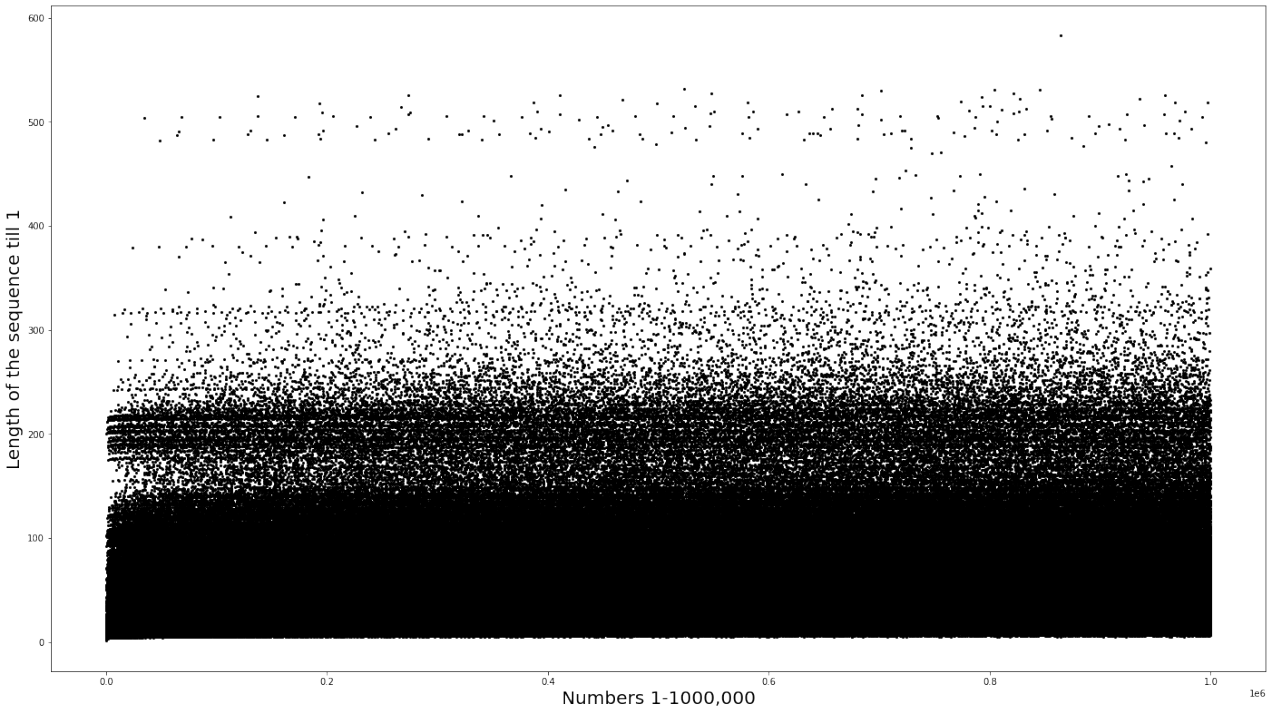


Figure 4.12: Number of terms up to 1, in the sequences generated by the $41n+1$ function using Python.

- Loop : $[1, 42, 21, 7, 1, ...]$

43n + 1 Sequence Lengths 43n + 1 Sequence loops

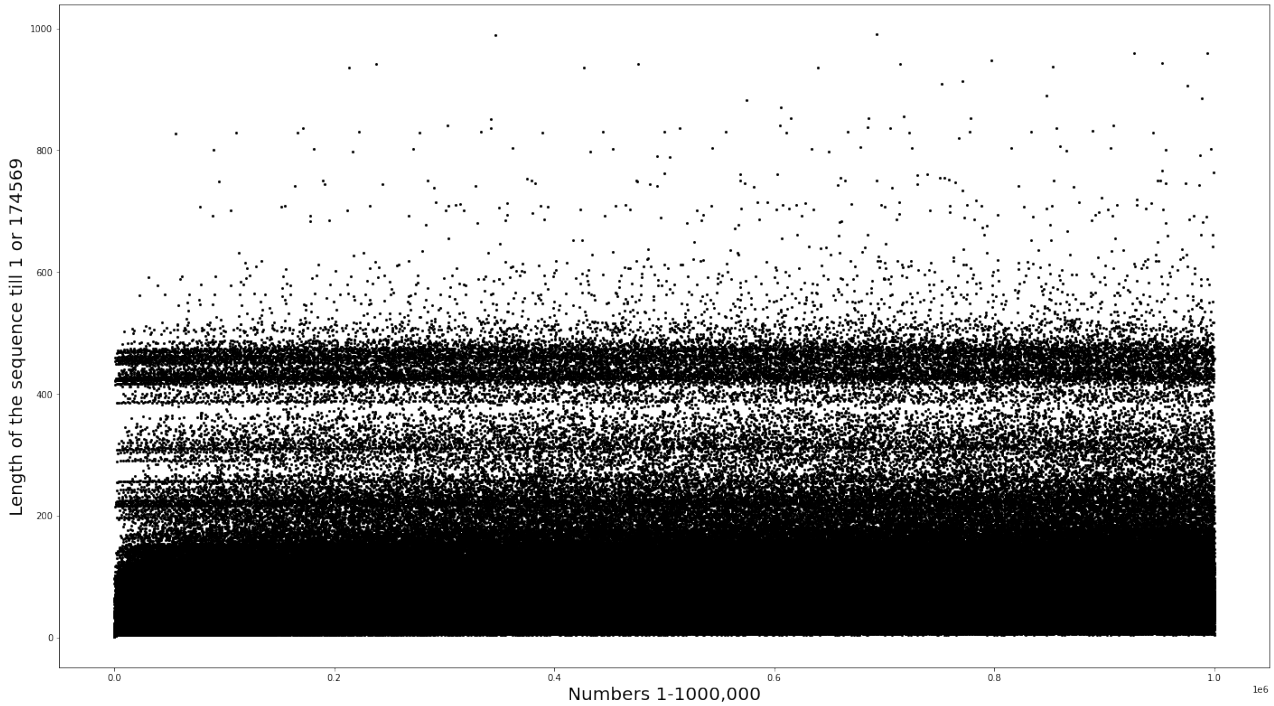


Figure 4.13: Number of terms up to a loop, in the sequences generated by the $43n+1$ function using Python.

- Loop 1 : [1, 44, 22, 11, 1, ...]
- Loop 2 : [174569, 7506468, 3753234, 1876617, 625539, 208513, 8966060, 4483030, 2241515, 448303, 19277030, 9638515, 1927703, 82891230, 41445615, 13815205, 2763041, 118810764, 59405382, 29702691, 9900897, 3300299, 141912858, 70956429, 23652143, 1017042150, 508521075, 169507025, 33901405, 6780281, 291552084, 145776042, 72888021, 24296007, 8098669, 348242768, 174121384, 87060692, 43530346, 21765173, 935902440, 467951220, 233975610, 116987805, 38995935, 12998645, 2599729, 236339, 10162578, 5081289, 1693763, 72831810, 36415905, 12138635, 2427727, 104392262, 52196131, 4015087, 174569, ...]

47n + 1 Sequence Lengths

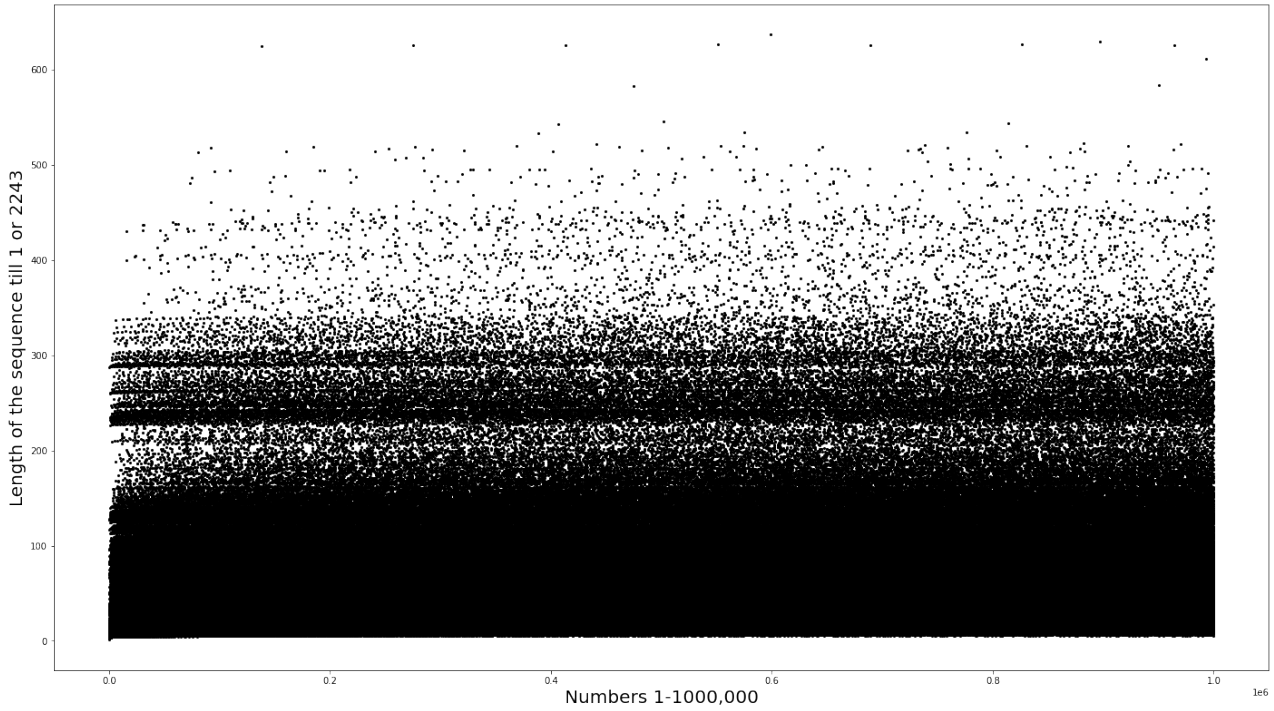


Figure 4.14: Number of terms up to a loop, in the sequences generated by the $47n+1$ function using Python.

47n + 1 Sequence loops

- Loop 1 : [1, 48, 24, 12, 6, 3, 1, ...]
- Loop 2 : [2243, 105422, 52711, 2477418, 1238709, 412903, 19406442, 9703221, 3234407, 294037, 13819740, 6909870, 3454935, 1151645, 230329, 20939, 984134, 492067, 23127150, 11563575, 3854525, 770905, 154181, 7246508, 3623254, 1811627, 85146470, 42573235, 8514647, 400188410, 200094205, 40018841, 1880885528, 940442764, 470221382, 235110691, 6354343, 171739, 8071734, 4035867, 1345289, 122299, 5748054, 2874027, 958009, 73693, 3463572, 1731786, 865893, 288631, 41233, 1937952, 968976, 484488, 242244, 121122, 60561, 20187, 6729, 2243, ...]

4.1.2 Pn + 1 functions with Pell prime numbers

The Pell numbers

The Pell numbers[18] are defined by the recurrence relation,

$$P_n = \begin{cases} 0, & \text{If } n = 0 \\ 1, & \text{If } n = 1 \\ 2P_{n-1} + P_{n-2}, & \text{Otherwise.} \end{cases}$$

for $n \in \mathbb{N}$.

The Companion Pell numbers are defined by the recurrence relation,

$$P_n = \begin{cases} 1, & \text{If } n = 0 \\ 1, & \text{If } n = 1 \\ 2P_{n-1} + P_{n-2}, & \text{Otherwise.} \end{cases}$$

for $n \in \mathbb{N}$.

- We can obtain Pell numbers using the recurrence relation, which are $0, 1, 2, 5, 12, 29, 70, 169, 408, 985, 2378, 5741, ...$ and we get Pell prime numbers as $2, 5, 29, 5741, 33461, 44560482149, ...$
- $Pn + 1$ sequences with Pell prime numbers and corresponding companion Pell primes, have only 1 loop.
  - This is true for first four *Pell Primes*: $2, 5, 29, 5741$.
  - This is true for Corresponding *Companion Pell primes*: $3, 7, 41$.

4.2 Encryption and Decryption algorithm using the Collatz sequence

In the present cryptographic scheme, we use the Collatz sequence to encrypt a plaintext symmetrically. We have proposed a new cryptographic system in which the sender and the receiver hold the secret key pair (k, l) which has been chosen randomly. A plaintext is encrypted using a large, odd number k and l number of iterations in the Collatz sequence starting with k.

4.2.1 Encryption Algorithm

Suppose we want to encrypt some random plaintext M, with r number of characters and the secret key (k, l) is chosen randomly.

- **Step 1:** First, we convert each character of M into 3-digit ASCII representation, such that we get $3r$ digit length code.
- **Step 2:** In this step, we need to make l number of partitions from the $3r$-digit ASCII code. If $3r$ is not divisible by l, we pad t number of zeros in front of the code, such that $3r + t$ is divisible by l, for some positive integer t. Thus, $3r + t$ digits can partition into l partitions with $(3r + t)/l$-digit lengths.
  - We denote the partitioned ASCII code as $[m_1, m_2, ...m_l]$.
- **Step 3:** Then, we calculate the Collatz sequence of k.
- **Step 4:** After that, from the sequence we generated, We obtain first l consecutive odd numbers, which are denoted as $[k_0, k_1, ..., k_l]$ and the even number count between those numbers, which are denoted as $[n_1, n_2, ..., n_l]$.
- **Step 5:** Each n_i is padded in front of each m_i for $i = 1, 2, ..., l$.
  - Partitioned and padded ASCII code denoted as $[n_1m_1, n_2m_2, ..., n_lm_l]$

- **Step 6:** Finally, the partitions we created in the last step are taken together as a one number, which is denoted as M' and we multiply that number with l^{th} consecutive odd number, which is k_l.
  - The encrypted message is $M' \cdot k_l = [(n_1 m_1 n_2 m_2 ... n_l m_l) \cdot k_l]$

Illustration using Python

Suppose we take the the plain text as 'mathematics', which is an 11-letter word and the secret key (1111,5) is chosen randomly.

- **Step 1:** First we convert each letter into 3-digit ASCII representation such that we get $33 (= 11 \cdot 3)$ digit code.
  - ASCII representations of 'mathematics' are $[109, 097, 116, 104, 101, 109, 097, 116, 105, 099, 115]$ such that [10909711610410110909711610509911 5] is the 33 digit code.
- **Step 2:** In this step, we need to make $l\,(= 5)$ number of partitions from the 33-digit ASCII code. Since 33 is not divisible by 5, we pad two zeros in front of the code. Thus, $35\,(33 + 2)$ digits can partition into 5 partitions with 7-digit lengths.
  - Partitioned ASCII code is $[m_1, m_2, m_3, m_4, m_5] = [0010909, 7116104, 1011090, 9711610, 5099115]$
- **Step 3:** Then, we calculate the Collatz sequence of 1111, which is $[1111, 3334, 1667, 5002, 2501, 7504, 3752, 1876, 938, 469, 1408, 704, 352, 176, 88, 44, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1]$.
- **Step 4:** After that, from the sequence we generated, We obtain first 5 consecutive odd numbers, which are $[1667, 2501, 469, 11, 17]$ and the even number count between those 5 consecutive odd numbers, which are $[1, 1, 4, 7, 1]$.
- **Step 5:** Each n_i is padded in front of each m_i for $i = 1, 2, 3, 4, 5$.
  - Partitioned and padded ASCII code is $[10010909, 17116104, 41011090, 79711610, 15099115]$.

- **Step 6:** Finally, the partitions we created in the last step are taken together as a one number and we multiply that number with $l^{th}(= 5^{th})$ consecutive odd number which is 17.

  - The encrypted message is $M' \cdot k_5 = [1001090917116104410110907971161015099115 \cdot 17]$ and denoted by $C = [317018545590973774971885435509737256684955]$.

4.2.2 Decryption Algorithm Using the Collatz sequence

Suppose we want to decrypt the encrypted message, C using the shared Secret key, (k, l) for some odd number k, and l number of iterations in the Collatz sequence, starting with k.

- **Step 1:** First we generate the Collatz sequence starting with k. So, we can obtain first n consecutive odd numbers, which are $[k_1, k_2, ..., k_l]$ and the even number count between those l consecutive odd numbers which are $[n_1, n_2, ..., n_l]$.

- **Step 2:** Next, we divide C by l^{th} consecutive odd number, which is k_l. Then we get $M' = C/k_l$.

- **Step 3:** After that, we need to remove the padded n_i's. Since we know the number of digits in M' and l of them are n_is, we can get l partitions without them. Thus, we get the partitioned ASCII code as $[n_1m_1, n_2m_2, ..., n_lm_l]$.

- **Step 4:** In this step, we take all l) partitions together as a one number and remove all the zeros in front of it. Thus, we can get 3-digit ASCII representations of the encrypted plaintext(M).

- **Step 5:** Finally, we can get the plaintext(M) using the ASCII table.

  - Decrypted text is denoted as $D(C) = M$

Illustration using Python

Suppose we receive the encrypted message as $C = [3170185455909737749718854355097372566849 55]$ and the shared Secret key is $(1111, 5)$.

- **Step 1:** First we generate the Collatz sequence of 1111. So, we can obtain first 5 consecutive odd numbers, which are $[1667, 2501, 469, 11, 17]$ and the even number count between those 5 consecutive odd numbers which are $[1, 1, 4, 7, 1]$. Thus, $n_1 = 1$, $n_2 = 1$, $n_3 = 4$, $n_4 = 7$, $n_5 = 1$ and $k_5 = 17$.

- **Step 2:** Next, we divide C by $l^{th}(= 5^{th})$ consecutive odd number, which is 17. Then we get $M' = C/k_5 = [1001090917116104410110907971161015099115]$

- **Step 3:** After that, we need to remove the padded n_i's. Since there are is a 40 digits in M' and 5 of them are n_is, we can get 5 partitions without them. Thus, we get the partitioned ASCII code as $[0010909, 7116104, 1011090, 9711610, 5099115]$.

- **Step 4:** In this step, we take all $l(= 5)$ partitions together as a one number and remove all the zeros in front of it. Thus, we can get 3-digit ASCII representations of the encrypted plaintext(M), which are $[109, 097, 116, 104, 101, 109, 097, 116, 105, 099, 115]$.

- **Step 5:** Finally, we can get the plaintext(M) using the ASCII table.

  - Decrypted text is denoted as $D(C) =$ 'mathematics'.

Chapter 5

Conclusions

5.1 The approach on the Collatz conjecture

We approached the Collatz conjecture in a constructive way. We know that the Collatz sequence is conjectured for any positive integer n and all the even numbers of the form $n = k \cdot 2^m$ (for some odd number k and positive integer m) converge to the odd number k, through a sequence generated by the Collatz function. Thus, we only need to prove the Collatz conjecture is true for all odd positive integers.

Also, we have showed that for any level in the Collatz structure, odd numbers reside in a sequence of the form $[k] = \left\{k_1' \cdot 4^{n-1} + \sum_{i=0}^{n-2} 4^i\right\}_{n=1}^{\infty}$ and if k_1' is convergent to 1, then all the terms on the sequence of the form $k_1' \cdot 4^{n-1} + \sum_{i=0}^{n-2} 4^i$ for all n are convergent to 1, through a sequence generated from the odd Collatz function $f(f(k') = k)$. Thus, we only need to prove the Collatz conjecture is true for all the odd k_1' numbers.

On the Collatz structure the Collatz sequence converges upwards and we have defined a function $g(g(k) = k_1'$ and $g(k) = 4k + 1$ for $k \equiv 0 \pmod 3)$, which diverges downward. Even though the function g is not the exact inverse of the odd Collatz function $f(f(k') = k)$, if $g^t(k) = k_t$ for some positive integer t and odd numbers k and k_t, we have $f^{t'}(k_t) = k$ for some $t'(\leq t)$ such that we can go from k_t to k using the odd Collatz function. Hence for every divergent sequence generated from the function g, we have a convergent Collatz sequence generated from

the function f, starting with a term of the sequence generated by g and true for any term of the sequence generated by g. Thus, our present study provides a good justification for why the Collatz conjecture holds true for all positive integers.

5.2 Similar sequences to the Collatz sequence

We used Python to generate $Pn + 1$ sequences starting from the numbers 1 to 1000,000 and we checked for the numbers which are repeating in a generated sequence and number of terms it takes to reach a loop(length). Since all the sequences had a finite length, we can conclude that all $Pn + 1$ sequences are convergent for $P < 50$, but the sequences which are similar to the Collatz sequence, have only one loop. For the first 4 Pell-prime numbers, we had only one loop and also, since there are infinitely many Pell-prime numbers, there can be infinitely many similar sequences to the Collatz sequence.

5.3 The cryptographic scheme based on the Collatz sequence

In the present cryptographic scheme, we use the Collatz sequence to encrypt a plaintext symmetrically. We have proposed a new cryptosystem in which the sender and the receiver hold the secret key pair (k, l) which has been chosen randomly. A plaintext is encrypted using a large, odd number k, and l number of iterations in the Collatz sequence, starting with k.
The Collatz sequence is a chaos-based[16] sequence and for every positive integer there is a unique mapping, and a slight change of the number makes a huge difference in the generated sequence. Thus, the chaotic nature makes this cryptosystem more secure. This cryptosystem has uncomplicated transformations which are very simple to be analyzed and is efficient because the only operations involved are multiplication and addition. It can be used to build more secure cryptosystems by repeating the encryption process or implementing it with other cryptosystems.

Appendices

Python Encryption Algorithm

```
collseed = 1111
lenn = 5
message_to_enc='mathematics'

def collatz(number,n):
  count = 0
  evencount = 0
  lst=[]
  oddll = []
  lst.append(number)
  while(number!=1):
      if(number%2==0):
          number=number//2
          lst.append(number)
          # if count < n:
      else:
          oddll.append(number)
          number=number*3+1
          lst.append(number)
      count = count+1

      if(len(oddll)==n+1):
        return [lst,oddll]
  print(lst)
  return [lst,oddll]

cooll = collatz(collseed,lenn)
ncolatz = cooll[0]
oddcolatz = cooll[1]
nthoddcolatz = cooll[1][lenn]

evencount = []
evencountpass = 0
for ev in ncolatz[1:]:
  if ev%2 == 0:
    evencountpass = evencountpass+1
  else:
```

```
    evencount.append(evencountpass)
    evencountpass = 0

print(cooll)
print(ncolatz)
print(oddcolatz)
print(nthoddcolatz)
print(evencount)
def strtoascii(srting):
  strlist = [ord(c) for c in srting]
  intll = []
  for i in strlist:
    n = str(i)
    filll = n.zfill(3)
    intll.append(filll)

  longstr = "".join(intll)
  print(longstr)
  print(intll)

  return longstr
longstr = strtoascii(message_to_enc)
lenmod = len(longstr)%lenn

print(lenmod)

if(lenmod!=0):
  paddedlstr = longstr.zfill(len(longstr)+(lenn-lenmod))
else:
  paddedlstr = longstr

print(paddedlstr)
print(len(paddedlstr))
partition_size = len(paddedlstr)//lenn
print(partition_size)

out = [(paddedlstr[i:i+partition_size]) for i in range(0, len(paddedlstr)
    , partition_size)]

print(out)
encylst = []
for idx , nthsss in enumerate(out):
  encccc = str(evencount[idx]%10)+nthsss
  # encccc = int(nthsss)+oddcolatz[idx]
  encylst.append(encccc)

print(encylst)
longencystr = "".join(encylst)
encmulssint = int(longencystr)*nthoddcolatz

print(encmulssint)
```

Python Decryption Algorithm

```
decstr = '17018545590973774971881154355011973725668495512321 1'
collseed = 1111
lenn = 5

def collatz(number,n):
  count = 0
  lst=[]
  oddll = []
  lst.append(number)
  while(number!=1):
      if(number%2==0):
          number=number//2
          lst.append(number)
          # if count < n:
      else:
          oddll.append(number)
          number=number*3+1
          lst.append(number)
      count = count+1

      if(len(oddll)==n+1):
        return [lst,oddll]
  print(lst)
  return [lst,oddll]

cooll = collatz(collseed,lenn)
ncolatz = cooll[0]
oddcolatz = cooll[1]
nthoddcolatz = cooll[1][lenn]

evencount = []

evencountpass = 0
for ev in ncolatz[1:]:
  if ev%2 == 0:
    evencountpass = evencountpass+1
  else:
    evencount.append(evencountpass)
    evencountpass = 0

print(cooll)
print(ncolatz)
print(oddcolatz)
print(nthoddcolatz)
print(evencount)

intmsg = int(decstr)//nthoddcolatz
print(intmsg)
strmsg = str(intmsg)
```

```
partition_size = len(strmsg)//lenn

split_list=list(map(''.join, zip(*[iter(strmsg)]*partition_size)))

print(split_list)

encoded_list = []
for item in split_list:
  encoded_list.append(item[1:])

print(encoded_list)

combine_str  = ''.join(encoded_list).lstrip('0')
# combine_str =
print(combine_str)

decoded_ascii_list = []
if len(combine_str)%3 == 0:
  decoded_ascii_list = list(map(''.join, zip(*[iter(combine_str)]*3)))
else:
  decoded_ascii_list = list(map(''.join, zip(*[iter('0'+combine_str)]*3))
     )

print(decoded_ascii_list)

decoded_char_list = []
for item in decoded_ascii_list:
  decoded_char_list.append(chr(int(item)))

decoded_msg = ''.join(decoded_char_list)

print(decoded_msg)
```

Python codes for the number of steps in the Pn + 1 Sequence

Python code for Prime numbers

```
def getprimes(n):
  primelist = []
  for num in range(2,n):
      prime = True
      for i in range(2,num):
          if (num%i==0):
              prime = False
      if prime:
        # print (num)
        primelist.append(num)
  return primelist
```

Python code for the repetitions occurred in the Pn + 1 sequence

def collatzwprime(x,primes): initial = x a = [x] repeats = []

```
 while x != 1:
      nner = False
      # print(x)
      for item in primes:
        if x % item == 0:
          x = x/item
          nner = True
          break
      if nner is not True:
        if x in repeats:
          print(initial,x)
          break
        repeats.append(x)
        x = P*x + 1
      a.append(x)
    return(a)
```

Python code for Number of steps graph using matplot library

```
import matplotlib.pyplot as plt
x=[]
y=[]
for i in range(1,1000000):
    a = collatzwprime(i,getprimes(P))
    x.append(i)
    y.append(len(a))
plt.figure(figsize=(24,13.5))
plt.xlabel("Numbers 1-1000,000",fontsize=20)
plt.ylabel("Length of the sequence till 1",fontsize=20)
plt.plot(x,y,'ko', markersize=2)
plt.show()
```

Bibliography

[1] Monika Agrawal and Pradeep Mishra. A comparative survey on symmetric key encryption techniques. *International Journal on Computer Science and Engineering*, 4(5):877, 2012.

[2] Ethan Akin. *The general topology of dynamical systems*, volume 1. American Mathematical Soc., 1993.

[3] Ştefan Andrei and Cristian Masalagiu. About the collatz conjecture. *Acta Informatica*, 35(2):167–179, 1998.

[4] Abdullah N Arslan. Methods for constructing collatz numbers. *Notes on Number Theory and Discrete Mathematics*, 24(2):47–54, 2018.

[5] Dora M Ballesteros, Jimmy Peña, and Diego Renza. A novel image encryption scheme based on collatz conjecture. *Entropy*, 20(12):901, 2018.

[6] Van Bendegem and Jean Paul. The collatz conjecture. a case study in mathematical problem solving. *Logic and Logical Philosophy*, 14(1):7–23, 2005.

[7] Marc Chamberland. A 3x+ 1 survey: Number theory and dynamical systems. *The ultimate challenge: the 3x*, 1:57–78, 2010.

[8] J Conway. *Unpredictable iterations*. AMS Providence, RI, USA, 1972.

[9] John H Conway. Fractran: A simple universal programming language for arithmetic. In *Open problems in Communication and Computation*, pages 4–26. Springer, 1987.

[10] Matthew Cook, David Soloveichik, Erik Winfree, and Jehoshua Bruck. Programmability of chemical reaction networks. In *Algorithmic bioprocesses*, pages 543–584. Springer, 2009.

[11] Richard Guy. *Unsolved problems in number theory*, volume 1. Springer Science & Business Media, 2004.

[12] Kevin Hartnett. Mathematician proves huge result on dangerous problem. *Quanta Magazine*, 2019.

[13] Michael A Idowu. A novel theoretical framework formulated for information discovery from number system and collatz conjecture data. *Procedia Computer Science*, 61:105–111, 2015.

[14] Stephen D Isard and Arnold M Zwicky. Three open questions in the theory of one-symbol smullyan systems. *ACM SIGACT News*, (7):11–19, 1970.

[15] Atul Kahate. *Cryptography and network security*. Tata McGraw-Hill Education, 2013.

[16] Ljupco Kocarev. Chaos-based cryptography: A brief overview. *Circuits and Systems Magazine, IEEE*, 1:6 – 21, 2002.

[17] Alex V Kontorovich and Jeffrey C Lagarias. Stochastic models for the 3x+ 1 and 5x+ 1 problems and related problems. *The Ultimate Challenge: The 3x+ 1 Problem*, pages 131–188, 2010.

[18] Thomas Koshy. *Pell and Pell-Lucas numbers with applications*, volume 431. Springer, (2014).

[19] Jeffrey C Lagarias. The 3 x+ 1 problem and its generalizations. *The American Mathematical Monthly*, 92(1):3–23, 1985.

[20] Jeffrey C Lagarias. The $3x+1$ problem and its generalizations. *The American Mathematical Monthly*, 92(1):3–23, 1985.

[21] Jeffrey C Lagarias. *The ultimate challenge: The 3x+ 1 problem*. American Mathematical Soc., 2010.

[22] Kelly S LaTourette et al. *Explorations of the collatz conjecture*. Citeseer, 2007.

[23] Hans Lausch and Wilfried Nöbauer. Algebra of polynomials. north-holl. publ. co. *American Elsevier Publ. Co*, 39:70–72, 1973.

[24] Haoyu Ma, Chunfu Jia, Shijia Li, Wantong Zheng, and Dinghao Wu. Xmark: dynamic software watermarking using collatz conjecture. *IEEE Transactions on Information Forensics and Security*, 14(11):2859–2874, 2019.

[25] Keith R Matthews. Generalized 3x+ 1 mappings: markov chains and ergodic theory. *The ultimate challenge: The 3x*, 1:79–103, 2010.

[26] Pascal Michel and Maurice Margenstern. Generalized 3x+ 1 functions and the theory of computation. *The Ultimate Challenge: The 3x*, 1:105–130, 2010.

[27] Marvin Minsky. Computation: Finite and infinite machines prentice hall. *Inc., Engelwood Cliffs, NJ*, 1967.

[28] Marvin L Minsky. Recursive unsolvability of post's problem of" tag" and other topics in theory of turing machines. *Annals of Mathematics*, pages 437–455, 1961.

[29] Tomás Oliveira e Silva. Empirical verification of the $3x + 1$ and related conjectures. In Jeffrey C. Lagarias, editor, *The Ultimate Challenge: The* $3x + 1$ *Problem*, pages 189–207. American Mathematical Society, Providence, Rhode Island, USA, 2010.

[30] Wei Ren. A new approach on proving collatz conjecture. *Journal of Mathematics*, 2019, 2019.

[31] Wei Ren. Reduced collatz dynamics data reveals properties for the future proof of collatz conjecture. *Data*, 4(2):89, 2019.

[32] Diego Renza, Sebastián Mendoza, et al. High-uncertainty audio signal encryption based on the collatz conjecture. *Journal of Information Security and Applications*, 46:62–69, 2019.

[33] K Saranya, R Mohanapriya, and J Udhayan. A review on symmetric key encryption techniques in cryptography. *International Journal of Science, Engineering and Technology Research (IJSETR)*, 3(3):539–544, 2014.

[34] Bruce Schneier. *Applied cryptography: protocols, algorithms, and source code in C*. john wiley & sons, 2007.

[35] David Soloveichik, Matthew Cook, Erik Winfree, and Jehoshua Bruck. Computation with finite stochastic chemical reaction networks. *natural computing*, 7(4):615–633, 2008.

[36] William Stallings. *Data and computer communications*. Pearson Education India, 2007.

[37] Terence Tao. Almost all orbits of the collatz map attain almost bounded values. In *Forum of Mathematics, Pi*, volume 10. Cambridge University Press, 2022.

[38] Verzweigungsdiagramme und Hypergraphen. International series for numerical mathematics, vol. 38, 1977.

[39] Giovanni Venturini. On a generalization of the 3x+ 1 problem. *Advances in Applied Mathematics*, 19(3):295–305, 1997.

[40] GJ Wirsching. The dynamical system generated by the 3n+ 1, 1998.

[41] Roger Zarnowski. A refinement of the $3x+1$ conjecture. *arXiv preprint arXiv:1908.00311*, 2019.